

- Una clase plantilla se produce especificando el tipo de la clase que sigue al nombre de la clase, como sigue:

```
ClassName<type> objectName;
```

El compilador sustituirá **type** a todo lo largo de la definición de clase y de la definición de la función miembro correspondiente por el tipo parametrizado de la clase.

### Terminología

|                                                   |                                                  |
|---------------------------------------------------|--------------------------------------------------|
| operador de resolución de alcance binario ( : : ) | iterador                                         |
| llamadas de función de miembro encadenado         | operador de selección de miembro ( . )           |
| alcance de clase                                  | especificadores de acceso de miembro             |
| composición                                       | inicializador de miembro                         |
| función miembro <b>const</b>                      | objeto miembro                                   |
| objeto <b>const</b>                               | constructor de objeto miembro                    |
| constructor                                       | clase anidada                                    |
| clases contenedor                                 | operador <b>new</b>                              |
| constructor por omisión                           | tipo parametrizado                               |
| destructor por omisión                            | operador selector de miembro de apuntador ( -> ) |
| operador <b>delete</b>                            | apuntador a miembro                              |
| operador <b>delete</b> [ ]                        | principio de mínimo privilegio                   |
| destructor                                        | miembro de datos estático                        |
| objetos dinámicos                                 | función miembro estática                         |
| extensibilidad                                    | clase plantilla                                  |
| clase <b>friend</b>                               | apuntador <b>this</b>                            |
| función <b>friend</b>                             |                                                  |

### Errores comunes de programación

- 17.1 Definir como **const** una función miembro que modifique un miembro de datos de un objeto.
- 17.2 Definir como **const** una función miembro que llama a una función miembro no **const**.
- 17.3 Llamar a una función miembro no **const** en relación con un objeto **const**.
- 17.4 Intentar modificar un objeto **const**.
- 17.5 No proporcionar un inicializador de miembro para un objeto miembro **const**.
- 17.6 No proporcionar un constructor por omisión para un objeto miembro, cuando no se proporciona inicializador miembro para dicho objeto miembro. Esto puede dar como resultado un objeto miembro no inicializado.
- 17.7 Intentar utilizar el operador de selección de miembro ( . ) con un apuntador a un objeto (el operador de selección de miembro sólo puede ser utilizado con un objeto o con una referencia a un objeto).
- 17.8 Mezclar asignación dinámica de memoria del tipo **new** y **delete** con asignación dinámica de memoria del tipo **malloc** y **free**: el espacio creado por **malloc** no podrá ser liberado por **delete**; los objetos creados mediante **new** no podrán ser borrados por **free**.
- 17.9 Referirse al apuntador **this** desde dentro de una función miembro estática.
- 17.10 Declarar una función miembro estática **const**.

### Prácticas sanas de programación

- 17.1 Declare como **const** todas las funciones miembro que se pretenda utilizar con objetos **const**.
- 17.2 Coloque en la clase todas las definiciones de amistad en primer término, después del encabezado de clase, y no las anteceda con ningún especificador de acceso de miembros.

- 17.3 A pesar de que los programas C++ pueden contener almacenamiento creado por **malloc** y borrados por **free**, y objetos creados por **new** y borrados por **delete**, lo mejor es utilizar solo **new** y **delete**.

### Sugerencias de rendimiento

- 17.1 Inicialice de forma explícita los objetos miembro mediante inicializadores miembro. Esto elimina la sobrecarga de una doble inicialización de objetos miembro una vez cuando se llame al constructor por omisión del objeto miembro, y una segunda vez cuando se utilicen las funciones set para inicializar dicho objeto miembro.
- 17.2 Por razones de economía de almacenamiento, existe sólo una copia de cada función miembro por clase, y esta función miembro es invocada para todos los objetos de dicha clase. Por otra parte, cada objeto tiene su propia copia de los miembros de datos de la clase.
- 17.3 Cuando una copia de los datos sea suficiente, utilice miembros de datos estáticos a fin de ahorrar almacenamiento.

### Observaciones de ingeniería de software

- 17.1 Declarar un objeto como **const** ayuda a que se cumpla el principio del mínimo privilegio. Cualquier intento accidental de modificar dicho objeto será detectado en tiempo de compilación, en vez de que causar errores en tiempo de ejecución.
- 17.2 Una función miembro **const** puede ser homónima en una versión no **const**. El compilador selecciona de forma automática la función miembro homónima basándose en el objeto que ha sido declarado **const** o no.
- 17.3 Tanto los objetos **const** como las "variables" **const** necesitan ser inicializadas con sintaxis de inicializador miembro. Las asignaciones no son permitidas.
- 17.4 Una forma de reutilización del software es la composición, en la cual una clase tiene como miembros objetos de otras clases.
- 17.5 Si un objeto tiene varios objetos miembro, está indefinido el orden en el cual los objetos miembro serán construidos. No escriba código que dependa de que los constructores de objetos miembro ejecuten su trabajo en un orden específico.
- 17.6 Los conceptos de acceso de miembros correspondientes a **private**, **protected** y **public** no tienen relación con las declaraciones de amistad, por lo que las declaraciones de amistad pueden ser colocadas en cualquier parte de la definición de clase.
- 17.7 Algunas personas en la comunidad OOP (Programación orientada a objetos) sienten que la "amistad" corrompe el ocultamiento de la información y debilita el valor del enfoque de diseño orientado a objetos.
- 17.8 Los miembros de datos estáticos y las funciones miembro estáticas existen y pueden ser utilizados, aun si no se han producido objetos de dicha clase.
- 17.9 El programador puede crear nuevos tipos mediante el uso del mecanismo de clases. Estos nuevos tipos pueden ser diseñados para ser usados tan conveniente como los tipos incorporados. Por lo tanto; C++ resulta un lenguaje extensible. A pesar de que mediante estos nuevos tipos el lenguaje es fácil de ampliar, el lenguaje base mismo no es modificable.
- 17.10 Las clases plantilla fomentan la reutilización del software.

### Ejercicios de autoevaluación

- 17.1 Llene cada uno de los siguientes espacios en blanco:
  - a) Se utiliza sintaxis \_\_\_\_\_ para inicializar los miembros constantes de una clase.
  - b) En una clase debe declararse un \_\_\_\_\_ para tener acceso a los miembros de datos **private** de dicha clase.

- c) El operador \_\_\_\_\_ asigna dinámicamente memoria para un objeto de un tipo específico y regresa un \_\_\_\_\_ a dicho tipo.
- d) Un objeto constante debe de ser \_\_\_\_\_; no puede ser modificado después de creado.
- e) Un miembro de datos \_\_\_\_\_ representa información para todo el ámbito de la clase.
- f) Todo objeto mantiene un apuntador a sí mismo llamado el apuntador \_\_\_\_\_.
- g) \_\_\_\_\_ proporcionan un medio de describir el concepto genérico de una clase.
- h) La palabra reservada \_\_\_\_\_ especifica que un objeto o una variable no es modificable.
- i) Si no se proporciona un inicializador de miembro para un objeto miembro de una clase, se llama al \_\_\_\_\_ del objeto.
- j) Una función miembro puede ser **static** si no tiene acceso a los miembros de clase \_\_\_\_\_.
- k) Las funciones amigo pueden tener acceso a los miembros \_\_\_\_\_ y \_\_\_\_\_ de una clase.
- l) Las clases plantilla se conocen a veces como tipos \_\_\_\_\_.
- m) Los objetos miembro se construyen \_\_\_\_\_ que el objeto de clase que los contiene.
- n) El operador \_\_\_\_\_ recupera memoria asignada previamente por **new**.

17.2 Encuentre el error en cada uno de los siguientes y explique cómo corregirlo.

- a) 

```
char *string;
string = new char[20];
free(string);
```
- b) 

```
class Example {
public:
    Example(int y = 10) { data = y; }
    int getIncrementedData() const { return ++data; }
    static int getCount()
    {
        cout << "Data is " << data << endl;
        return count;
    }
private:
    int data;
    static int getCount;
};
```

### Respuestas a los ejercicios de autoevaluación

17.1 a) Inicializador de miembro. b) **friend**. c) **new**, apuntador. d) inicializado. e) **static**. f) **this**. g) clases plantilla. h) **const**. i) constructor por omisión. j) no **static**. k) **private**, **protected**. l) parametrizado. m) antes. n) **delete**.

17.2 a) Error: la memoria asignada dinámicamente por **new** es borrada por la función de biblioteca estándar de C **free**.

Corrección: utilice el operador **delete** de C++ para recuperar la memoria. La asignación dinámica de memoria de tipo C no debe de ser mezclada con los operadores de C++ **new** y **delete**.

b) Error: la definición de clase correspondiente a **Example** tiene dos errores. el primero ocurre en la función **getIncrementedData**. La función es declarada **const**, pero modifica el objeto. Corrección: Para corregir el primer error, elimine la palabra reservada **const** de la definición de **getIncrementedData**.

Error: el segundo error ocurre en la función **getCount**. Esta función es declarada **static**, por lo que no se le permite acceso a cualquier miembro no **static** de la clase.

Corrección: para corregir el segundo error, elimine la línea de salida de la definición de **getCount**.

### Ejercicios

17.3 Compare la asignación dinámica de memoria mediante los operadores de C++ **new** y **delete**, con la asignación dinámica de memoria utilizando las funciones de la biblioteca estándar de C **malloc** y **free**.

17.4 Explique el concepto de amistad en C++. Explique los aspectos negativos de la amistad, tal y como se describen en el texto.

17.5 ¿Puede una definición correcta de clase **Time** incluir ambos constructores siguientes?

```
Time (int h = 0, int m = 0, int s = 0);
Time ();
```

17.6 ¿Qué ocurre cuando un tipo de regreso, inclusive el tipo **void**, es especificado para un constructor o destructor?

17.7 Cree una clase **Date** con las siguientes capacidades:

a) Extraer la fecha en formatos múltiples como

```
DDD YYYY
MM/DD/YY
June 14, 1992
```

b) Utilizar constructores homónimos para crear objetos **Date** inicializados con fechas de los formatos de la parte (a).

c) Crear un constructor **Date** que lea la fecha del sistema utilizando las funciones estándar de biblioteca del encabezado **time.h** y que defina los miembros **Date**.

En el capítulo 18, podremos crear operadores para probar la igualdad de dos fechas y para comparar fechas, a fin de determinar si una fecha es anterior a otra o posterior a otra.

17.8 Utilizando como guía el programa de procesamiento de lista de la figura 12.3, construya una clase **List**. La clase deberá proporcionar las mismas capacidades que el ejemplo. Escriba un programa manejador para probar por completo dicha clase.

17.9 Utilizando como guía el programa de procesamiento de colas de la figura 12.13, construya una clase **Queue**. La clase deberá proporcionar las mismas capacidades que el ejemplo. Escriba un programa manejador para probar completamente dicha clase.

17.10 Utilizando como punto de partida la clase **List** desarrollada en el ejercicio 17.8, cree una clase plantilla **List**, que sea capaz de producir objetos **List** que contengan muchos tipos de datos diferentes. Escriba un programa manejador para probar completamente la clase plantilla.

17.11 Utilizando como punto de partida la clase **Queue** desarrollada en el ejercicio 17.9, cree una clase plantilla **Queue** que sea capaz de producir objetos **Queue** que contengan muchos tipos de datos diferentes. Escriba un programa manejador para probar por completo la clase plantilla.

# 18

---

## Homonimia de operadores

---

### Objetivos

- Comprender cómo redefinir operadores para que funcionen con nuevos tipos.
- Comprender cómo convertir objetos de una clase a otra clase.
- Aprender cuándo efectuar, y cuándo no efectuar, la homonimia de operadores.
- Estudiar varias clases interesantes que utilizan operadores homónimos.

*La diferencia entre construcción y creación es exactamente esto: que una cosa construida puede ser querida sólo después de construida; pero una cosa creada es querida antes de que exista.*

Gilbert Keith Chesterton

*Prefacio a Dickens, Pickwick Papers*

*El dado ha sido lanzado.*

Julius Caesar

*En realidad jamás nuestro doctor operaba, a menos de que fuese necesario. El era precisamente así. Si no necesitaba el dinero, no ponía una mano sobre usted.*

Herb Shriner

## Sinopsis

- 18.1 Introducción
- 18.2 Fundamentos de la homonimia de operadores
- 18.3 Restricciones sobre la homonimia de operadores
- 18.4 Funciones operador como miembros de clase en comparación con funciones amigo
- 18.5 Cómo hacer la homonimia de operadores de inserción y de extracción de flujo.
- 18.6 Homonimia de operadores unarios
- 18.7 Homonimia de operadores binarios
- 18.8 Estudio de caso: una clase Array
- 18.9 Conversión entre tipos
- 18.10 Estudio de caso: una clase String
- 18.11 Homonimia de ++ y --
- 18.12 Estudio de caso: una clase Date

*Resumen • Terminología • Errores comunes de programación • Prácticas sanas de programación • Sugerencias de rendimiento • Observaciones de ingeniería de software • Ejercicios de autoevaluación • Respuestas a los ejercicios de autoevaluación • Ejercicios.*

### 18.1 Introducción

En los capítulos 16 y 17, presentamos los fundamentos de las clases C++ y el concepto de tipos de datos abstractos (ADT). Las manipulaciones sobre los objetos de clase (es decir, sobre ejemplos de ADT) se llevaron a cabo enviando mensajes a los objetos (bajo forma de llamadas de funciones miembro). Para ciertos tipos de clases esta notación de llamada de función resulta engorrosa, en especial para clases matemáticas. Sería agradable, para estos tipos de clases, utilizar el extenso repertorio de operadores incorporados de C++ concebidos para especificar manipulaciones de objetos. En este capítulo, mostramos cómo habilitar a los operadores de C++ para funcionar con objetos de clase. Este proceso se conoce como homonimia de operador. Es un proceso directo y natural extender C++ con estas nuevas capacidades.

En C++ el operador << es utilizado para varios fines como operador de inserción de flujos y como operador de desplazamiento a la izquierda. Este es un ejemplo de *homonimia de operador*. Similar, >> también tiene su homónimo; es utilizado tanto como operador de extracción de flujo como operador de desplazamiento a la derecha. En la biblioteca de clases de C++, ambos operadores tienen homónimos. El lenguaje de C++ mismo hace la homonimia de + y de -. Estos operadores funcionan en forma diferente dependiendo del contexto en aritmética de enteros, aritmética de punto flotante y aritmética de apuntadores.

En general, C++ le permite al programador hacer homónimos de la mayor parte de los operadores, haciéndolos sensibles al contexto dentro del cual son utilizados. El compilador generará el código apropiado, basándose en la forma en la cual el operador sea utilizado. Algunos operadores se hacen homónimos en forma frecuente, en especial el operador de asignación y varios

operadores aritméticos, como + y -. La tarea llevada a cabo por los operadores homónimos, también pueden ser ejecutada mediante llamadas de función explícitas, pero la notación de operador es por lo general más legible.

Analizaremos cuándo utilizar homonimia de operadores y cuando no utilizarla. Mostraremos cómo hacer la homonimia de operadores, y presentaremos muchos programas completos utilizando operadores homónimos.

### 18.2 Fundamentos de la homonimia de operadores

La programación en C++ es un proceso sensible a los tipos y enfocado a los tipos. Los programadores pueden utilizar tipos incorporados y pueden definir nuevos tipos. Los tipos incorporados pueden ser utilizados aprovechando la extensa colección de operadores de C++. Los operadores proporcionan a los programadores una forma de notación concisa para expresar manipulaciones de objetos de tipos incorporados.

Los programadores pueden también utilizar operadores con tipos definidos por usuario. Aunque C++ no permite que se creen operadores nuevos, sí permite que se haga la homonimia de algunos operadores existentes, de tal forma que estos operadores sean utilizados con objetos de clase, teniendo los operadores un significado apropiado a los nuevos tipos. Esta es una de las características más poderosas de C++. La homonimia de operadores contribuye a la extensibilidad de C++, lo que seguro es uno de los atributos más llamativos de este lenguaje.

#### *Práctica sana de programación 18.1*

*Utilice la homonimia de operadores cuando ésta haga más claro un programa si efectúa las mismas operaciones mediante llamadas de función explícitas.*

#### *Práctica sana de programación 18.2*

*Evite un uso excesivo o inconsistente de la homonimia de operadores, ya que ello puede hacer que un programa resulte críptico o difícil de leer.*

A pesar de que la homonimia de operadores pudiera sonar como una capacidad exótica, la mayor parte de los programadores utilizan de forma implícita operadores homónimos en forma regular. Por ejemplo, el operador más (+) opera de forma bien diferente sobre enteros, flotantes y dobles. Pero, con todo, más (+) funciona bien con variables del tipo **int**, **float**, **double** y varios otros tipos incorporados, porque en el lenguaje C++ mismo se hace la homonimia del operador más (+).

Se hace la homonimia de operadores escribiendo una definición de función (con encabezado y cuerpo) como usted lo haría normalmente, excepto que ahora el nombre de la función se convierte en la palabra clave **operator**, seguida por el símbolo correspondiente al operador homónimo. Por ejemplo, para hacer el homónimo del operador de adición se utilizaría el nombre de función **operator+**.

Para utilizar un operador sobre objetos de clase, dicho operador *deberá* ser un homónimo con dos excepciones. El operador de asignación (=) puede ser utilizado sin homónimo con cualquier clase. Cuando no se incluye un operador de asignación homónimo, el comportamiento por omisión es una *copia a nivel de miembro* de los miembros de datos de la clase. Pronto veremos que dicha copia a nivel de miembro por omisión es peligrosa, tratándose de clases con miembros que apunten a almacenamiento dinámicamente asignado; para dichas clases por lo regular haremos la homonimia del operador de asignación. El operador de dirección (&) también puede ser utilizado sin homonimia con objetos de cualquier clase; sólo devuelve la dirección del objeto en memoria. También se puede hacer la homonimia de este operador.

La homonimia es lo más apropiado para clases matemáticas. A menudo éstas requieren que un conjunto substancial de operadores sean homónimos, para asegurar paralelismo con la forma en que estas clases matemáticas son manejadas en el mundo real. Por ejemplo, para una clase de números complejos sería poco usual hacer la homonimia de sólo la adición, dado que con números complejos también son muy utilizados otros operadores aritméticos.

C++ es un lenguaje rico en operadores. Los programadores de C++ comprenden el significado y el contexto de cada operador. Por lo tanto, cuando en relación con nuevas clases se trata de hacer la homonimia de operadores, los programadores probablemente harán selecciones razonables.

El punto de la homonimia de operadores es proporcionar las mismas expresiones concisas para tipos definidos por usuario, que las que C++ proporciona con su rica colección de operadores, en relación con tipos incorporados. La homonimia de operadores, sin embargo, no es automática; para llevar a cabo las operaciones deseadas el programador deberá escribir funciones de homonimia de operadores. Algunas veces estas funciones se realizan mejor como funciones miembro; algunas veces se realizan mejor como funciones amigo; y en ocasiones pueden ser hechas como funciones no miembro, y no amigo.

Un uso equivocado de la homonimia sería hacer la homonimia del operador + para llevar a cabo operaciones de tipo substracción o hacer la homonimia del operador / para ejecutar operaciones de tipo multiplicación. Un uso como éste de la homonimia puede hacer un programa confuso.

Práctica sana de programación 18.3

Haga la homonimia de operadores para llevar a cabo la misma función o funciones muy similares sobre objetos de clase que dichos operadores ejecutan sobre objetos de tipos incorporados.

Práctica sana de programación 18.4

Antes de escribir programas en C++ utilizando operadores homónimos, consulte los manuales de C++ correspondientes a su compilador, para informarse de las varias restricciones y requisitos únicos a operadores particulares.

18.3 Restricciones sobre la homonimia de operadores

Se puede hacer la homonimia de la mayor parte de los operadores de C++. Estos se muestran en la figura 18.1. En la figura 18.2 se muestran los operadores que no pueden tener homónimos.

La precedencia de un operador no puede ser modificada por la homonimia. Esto puede llevar a situaciones incómodas en las cuales un operador tiene un homónimo, pero de forma tal, que su precedencia fija es inapropiada. Sin embargo, se pueden utilizar paréntesis dentro de una expresión, a fin de obligar el orden de evaluación de operadores homónimos.

| Operadores que pueden tener homónimos |     |    |    |    |    |     |        |
|---------------------------------------|-----|----|----|----|----|-----|--------|
| +                                     | -   | *  | /  | %  | ^  | &   |        |
| ~                                     | !   | =  | <  | >  | += | -=  | *=     |
| /=                                    | %=  | ^= | &= | =  | << | >>  | >>=    |
| <<=                                   | ==  | != | <= | >= | && |     | ++     |
| --                                    | ->* | ,  | -> | [] | () | new | delete |

Fig. 18.1 Operadores que pueden tener homónimos.

| Operadores que no pueden tener homónimos |   |    |    |        |
|------------------------------------------|---|----|----|--------|
| .                                        | * | :: | ?: | sizeof |

Fig. 18.2 Operadores que no pueden tener homónimos.

La asociatividad de un operador tampoco puede ser modificada por la homonimia. Para hacer la homonimia de un operador no pueden utilizarse argumentos por omisión.

No es posible modificar el número de operandos que toma un operador: los operadores unarios homónimos se conservan como unarios; los operadores binarios homónimos se conservan como binarios. El único operador ternario de C++, (? :), no puede tener homónimo. Los operadores &, \*, + y - cada uno de ellos tienen versiones unaria y binaria; cada una de estas versiones unaria y binaria pueden tener homónimos en forma separada.

No es posible crear operadores nuevos; sólo se puede hacer la homonimia de operadores existentes. Esto impide que el programador utilice notaciones populares como el operador \*\*, utilizado en BASIC para la exponenciación.

Error común de programación 18.1

Intentar crear operadores nuevos.

El significado del funcionamiento de un operador sobre objetos de tipos incorporados no puede ser modificado por la homonimia de operadores. El programador no puede, por ejemplo, cambiar cómo + añade dos enteros. La homonimia de operador funciona sólo con objetos de tipos definidos por el usuario o por una combinación de un objeto de tipo definido por el usuario y un objeto de tipo incorporado.

Error común de programación 18.2

Intentar modificar cómo funciona un operador con objetos de tipos incorporados.

Observación de ingeniería de software 18.1

En una función operador, por lo menos un argumento debe ser un objeto de clase o una referencia a un objeto de clase. Esto impedirá que los programadores modifiquen cómo funcionan los operadores sobre objetos de tipos incorporados.

Al hacer la homonimia de (), [], ->, o = la función de homonimia de operador deberá ser declarada como un miembro de clase. Para los demás operadores, las funciones de homonimia de operador pueden ser amigo.

La homonimia de un operador de asignación y de un operador de adición, para permitir enunciados como

```
object2 = object2 + object1
```

no implica que el operador += quede también de forma automática homónimo para permitir enunciados como

```
object2 += object1;
```

Sin embargo; este comportamiento puede ser conseguido haciendo la homonimia explícita del operador `+=` correspondiente a dicha clase.

#### Error común de programación 18.3

*Suponer que hacer la homonimia de un operador (como `+`) hace automáticamente la homonimia de sus operadores relacionados (como `+=`). Los operadores sólo pueden tener homónimos en forma explícita; la homonimia implícita no existe.*

### 18.4 Funciones operador como miembros de clase en comparación con funciones amigo

Las funciones operador pueden ser funciones miembro o funciones no miembro; las funciones no miembro por lo regular son funciones amigo. Las funciones miembro utilizan el apuntador `this` en forma implícita, para obtener uno de sus argumentos de objeto de clase. En una llamada de función no miembro, este argumento de clase deberá estar listado en forma explícita.

Independiente que una función de operador esté puesta en práctica como función miembro o como función no miembro, aun así en expresiones, el operador será utilizado de la misma forma. Por lo tanto, ¿cuál de las puestas en práctica es la mejor?

Cuando una función operador es puesta en práctica como función miembro, el operando más a la izquierda (o único) deberá ser un objeto de clase (o una referencia a un objeto de clase) de la clase del operador. Si el operando izquierdo tiene que ser un objeto de clase diferente o de tipo incorporado, esta función de operador deberá ser puesta en práctica como función no miembro, exactamente como lo hicimos al hacer la homonimia de `<<` y `>>` como operadores de inserción y de extracción de flujo, respectivamente. Una función operador puesta en práctica como una función no miembro necesita ser un amigo, si dicha función debe tener acceso directo a miembros privados o protegidos de dicha clase.

El operador homónimo `<<` debe tener un operando izquierdo del tipo `ostream &` (como `cout` en la expresión `cout << classObject`), por lo que deberá ser función no miembro. De igual forma, el operador homónimo `>>` deberá tener un operando izquierdo del tipo `istream &` (como `cin` en la expresión `cin >> classObject`), de forma tal, que también él deberá de ser una función miembro. También, cada una de estas funciones de operador homónimas requiere tener acceso a los miembros de datos privados del objeto de clase que se está extrayendo o introduciendo, por lo que, por razones de rendimiento, esas funciones operador homónimas por lo regular se hacen funciones amigo de la clase.

#### Sugerencia de rendimiento 18.1

*Es posible hacer la homonimia de un operador como función no miembro, y no amigo, pero dicha función con la necesidad de tener acceso a datos privados o protegidos de una clase requeriría utilizar las funciones `set` o `get` proporcionadas en la interfaz pública de la clase. La sobrecarga de llamar a estas funciones causaría bajo rendimiento.*

Las funciones miembro operador son llamadas sólo cuando el operando izquierdo de un operador binario es de forma específica un objeto de dicha clase o cuando el único operando de un operador unario es un objeto de dicha clase.

Otra razón por la cual uno podría elegir una función no miembro para hacer la homonimia de un operador, sería para permitir que el operador fuera conmutativo. Por ejemplo, suponga que tenemos un objeto, `number`, del tipo `long int` y un objeto `bigInteger1`, de la clase `HugeInteger` (una clase en la cual los enteros pueden ser de manera arbitraria extensos, en vez de quedar limitados por el tamaño de palabra de máquina del hardware subyacente; en los

ejercicios del capítulo se desarrolla la clase `HugeInteger`). El operador de adición (`+`) produce un objeto temporal `HugeInteger` como la suma de un `HugeInteger` y de un `long int` (como en la expresión `bigInteger1 + number`), o como la suma de un `long int` y un `HugeInteger` (como en la expresión `number + bigInteger1`). Por lo tanto, necesitamos que el operador de adición sea conmutativo (exactamente como normalmente es). El problema estriba en que, si debe hacer la homonimia del operador como función miembro, el objeto de clase debe aparecer a la izquierda de la adición. Por lo tanto, hacemos la homonimia del operador como amigo, para permitir que `HugeInteger` aparezca a la derecha de la adición. La función `operator+`, que se ocupa de `HugeInteger` a la izquierda, aún puede ser una función miembro.

### 18.5 Cómo hacer la homonimia de operadores de inserción y de extracción de flujo

C++ tiene la capacidad de introducir y de extraer los tipos de datos estándar utilizando los operadores de extracción de flujo `>>` y de inserción de flujo `<<`. Se hace la homonimia de estos operadores (incluidos en las bibliotecas de clase proporcionadas con los compiladores C++) para procesar cada tipo de datos estándar, incluyendo cadenas y direcciones de memoria. Los operadores de inserción y de extracción de flujo también pueden tener homónimos para ejecutar entradas y salidas para tipos definidos por usuario. En la figura 18.3 se demuestra la homonimia de los operadores de extracción y de inserción de flujo para manejar datos de una clase de número de teléfono definida por usuario llamada `PhoneNumber`. Este programa supone que los números telefónicos están introducidos de forma correcta. Dejamos para los ejercicios la inclusión de la verificación de errores.

La función operador de extracción de flujo (`operator>>`) toma como argumentos una referencia `istream (input)` y una referencia (`num`) a un tipo definido por usuario (`PhoneNumber`), y devuelve una referencia `istream`. En la figura 18.3 se utiliza la función de operador `operator>>` para introducir números telefónicos de la forma

```
(800) 555-1212
```

en objetos de la clase `PhoneNumber`. Cuando el compilador ve en `main` la expresión

```
cin >> phone
```

genera la llamada de función

```
operator>>(cin, phone);
```

Cuando esta llamada se ejecuta, el parámetro `input` se convierte en un seudónimo para `cin` y el parámetro `num` se convierte en un seudónimo para `phone`. La función operador utiliza la función miembro `istream`, de nombre `getline`, para leer como cadenas las tres porciones del número telefónico a los miembros `areaCode`, `exchange` y `line` del objeto referenciado `PhoneNumber` (`num` en la función operador y `phone` en `main`). La función `getline` explica en detalle en el capítulo 21. Los caracteres paréntesis, espacio y guión son pasados por alto, llamando a la función miembro `istream`, de nombre `ignore`, que descarta el número especificado de caracteres del flujo de entrada (un carácter por omisión). La función `operator>>` devuelve la referencia `istream` de nombre `input` (es decir, `cin`). Esto permite que se concatenen operaciones de entrada sobre objetos `PhoneNumber` con operaciones de entrada sobre otros objetos `PhoneNumber` o sobre objetos de otros tipos de datos. Por ejemplo, se podrían introducir dos objetos `PhoneNumber` como sigue:

```
cin >> phone1 >> phone2;
```

```

// FIG18_3.CPP
// Overloading the stream-insertion and
// stream-extraction operators.
#include <iostream.h>

class PhoneNumber {
    friend ostream &operator<<(ostream &, const PhoneNumber &);
    friend istream &operator>>(istream &, PhoneNumber &);
private:
    char areaCode[4]; // 3-digit area code and null
    char exchange[4]; // 3-digit exchange and null
    char line[5];     // 4-digit line and null
};

// Overloaded stream insertion operator (cannot be
// a member function).
ostream &operator<<(ostream &output, const PhoneNumber &num)
{
    output << "(" << num.areaCode << " ) "
           << num.exchange << "-" << num.line;
    return output; // enables cout << a << b << c;
}

// overloaded stream extraction operator
istream &operator>>(istream &input, PhoneNumber &num)
{
    input.ignore(); // skip (
    input.getline(num.areaCode, 4); // input area code
    input.ignore(2); // skip ) and space
    input.getline(num.exchange, 4); // input exchange
    input.ignore(); // skip dash (-)
    input.getline(num.line, 5); // input line

    return input; // enables cin >> a >> b >> c;
}

main()
{
    PhoneNumber phone; // create object phone

    cout << "Enter a phone number in the "
         << "form (123) 456-7890:\n";

    // cin >> phone invokes operator>> function by
    // issuing the call operator>>(cin, phone).
    cin >> phone;

    // cout << phone invokes operator<< function by
    // issuing the call operator<<(cout, phone).
    cout << "The phone number entered was:\n"
         << phone << endl;
    return 0;
}

```

Fig. 18.3 Operadores de inserción y de extracción de flujo definidos por usuario (parte 1 de 2).

```

Enter a phone number in the form (123) 456-7890:
(800) 555-1212
The phone number entered was:
(800) 555-1212

```

Fig. 18.3 Operadores de inserción y de extracción de flujo definidos por usuario (parte 2 de 2).

Primero, se ejecutaría la expresión `cin >> phone1` mediante la llamada

```
operator>>(cin, phone1);
```

Esta llamada a continuación regresaría `cin` como el valor de `cin >> phone1`, por lo que la porción restante de la expresión se interpretaría solo como `cin >> phone2`.

El operador de inserción de flujo toma como argumentos una referencia `ostream` (output) y una referencia (`num`) a un tipo definido por usuario (`PhoneNumber`), y devuelve una referencia `ostream`. La función `operator<<` muestra objetos del tipo `PhoneNumber`.

Cuando en `main` el compilador ve la expresión

```
cout << phone
```

generará la llamada de función

```
operator<<(cout, phone);
```

La función `operator<<` muestra las porciones del número telefónico como cadenas, porque están almacenadas en formato de cadena (la función miembro `istream` de nombre `getline` almacena un carácter nulo después de que termina su entrada).

Note que las funciones `operator>>` y `operator<<` se declaran en `class PhoneNumber` como funciones no miembro, amigo. Estos operadores deben ser no miembro, porque en cada caso el objeto de la clase `PhoneNumber` aparece como el operando derecho del operador; a fin de hacer la homonimia del operador como función miembro —el operando de clase debe aparecer a la izquierda. Los operadores de entrada y de salida homónimos deben ser declarados como amigo, si han de tener acceso directo a los miembros de clase no públicos.

#### Observación de ingeniería de software 18.2

Se pueden añadir nuevas capacidades de entrada/salida a C++ correspondientes a tipos definidos por usuario, sin modificar las declaraciones o los miembros de datos privados correspondientes a la clase `ostream` o a la clase `istream`. Esto promueve la extensibilidad del lenguaje de programación C++ uno de los aspectos más atractivos de C++.

## 18.6 Homonimia de operadores unarios

Se puede hacer la homonimia de un operador unario para una clase como función miembro no estática sin argumentos, o como función no miembro con un argumento; dicho argumento debe ser un objeto de la clase o una referencia a un objeto de la clase.

Más adelante en este capítulo haremos la homonimia del operador unario `!` para probar si una cadena está vacía. Al hacer la homonimia de un operador unario como `!`, en forma de función miembro no estática sin argumentos, si `s` es un objeto de clase, cuando el compilador vea la expresión `!s` generará la llamada `s.operator!()`. El operando `s` es el objeto de clase `s` para el cual se está invocando la función miembro `operator!`:

```
class String {
public:
    int operator!() const;
    ...
};
```

Existan dos formas distintas en que se puede hacer la homonimia de un operador unario como `!` en forma de función no miembro con un argumento —con un argumento que es un objeto o con un argumento que es una referencia a un objeto. Si `s` es un objeto de clase, entonces `!s` se tratará como si se hubiera escrito la llamada `operator!(s)`.

```
class String {
    friend int operator!(const String &);
    ...
};
```

#### Práctica sana de programación 18.5

*Al hacer la homonimia de operadores unarios, es preferible hacer la funciones operador miembros de clase en vez de funciones amigo no miembro. Esta es una solución más limpia. Las funciones amigo y las clases amigo deberán evitarse, salvo que sean en lo absoluto necesarias. La utilización de amigos viola el encapsulado de una clase.*

### 18.7 Homonimia de operadores binarios

Se puede hacer la homonimia de un operador binario como función miembro no estática con un argumento o como, función no miembro con dos argumentos (uno de dichos argumentos debiendo ser un objeto de clase o una referencia a un objeto de clase).

Más adelante en este capítulo, haremos la homonimia de `+=` para indicar concatenación de dos objetos de cadena. Al hacer la homonimia del operador binario `+=` como una función miembro no estática de una clase `String` con un argumento, si `y` y `z` son objetos de clase, entonces `y += z` será tratado como si se hubiera escrito `y.operator+=(z)`.

```
class String {
public:
    String &operator+=(const String &);
    ...
};
```

Se puede hacer la homonimia del operador binario `+=` como función no miembro con dos argumentos uno de los cuales debe ser un objeto de clase o una referencia a un objeto de clase. Si `y` y `z` son objetos de clase, entonces `y += z` será tratado como si dentro del programa la llamada `operator+=(y, z)` hubiera sido escrita

```
class String {
    friend String &operator+=(String &, const String &);
    ...
};
```

### 18.8 Estudio de caso: una clase Array

En C y en C++ la notación de arreglo es solo una alternativa a los apuntadores, por lo que los arreglos tienen mucho potencial para errores. Por ejemplo, un programa con facilidad puede “salirse” de un arreglo, porque C y C++ no verifican si los subíndices caen fuera del rango de un arreglo. Los arreglos del tamaño  $n$  deberá numerar sus elementos  $0, \dots, n - 1$ ; no se permiten rangos con subíndices distintos. Un arreglo completo no puede ser introducido o extraído de una vez; cada elemento del arreglo debe ser leído o escrito en forma individual. No se pueden comparar dos arreglos mediante operadores de igualdad u operadores relacionales. Cuando se pasa un arreglo a una función de uso general, diseñada para manejar arreglos de cualquier tamaño, como argumento adicional deberá pasarse el tamaño del arreglo. Un arreglo no puede ser asignado a otro arreglo utilizando el operador u operadores de asignación. Estas y otras capacidades ciertamente parecerían como “naturales” para el manejo de arreglos, pero C y C++ no tienen estas capacidades. Sin embargo, C++ sí proporciona los medios para poner en práctica estas capacidades para arreglos, mediante los mecanismos de la homonimia de operadores.

En este ejemplo, desarrollamos una clase de arreglo que lleva a cabo verificación de rangos, con el fin de asegurarse que los subíndices se conservan dentro de los límites del arreglo. La clase permite que, mediante el operador de asignación, un objeto de arreglo sea asignado a otro. Los objetos de la clase de arreglo de forma automática conocen su tamaño, por lo que al pasar el arreglo a una función dicho tamaño no necesita ser pasado como argumento. Mediante los operadores de extracción y de inserción de flujo, respectivamente, es posible introducir o extraer arreglos completos. Se pueden llevar a cabo comparaciones de arreglos utilizando los operadores de igualdad `==` y `!=`. Nuestra clase de arreglo utiliza un miembro estático para llevar control del número de objetos de arreglo que en el programa han sido producidos. Este ejemplo aumentará su aprecio a la abstracción de datos. Es probable que desee sugerir varias mejoras a esta clase de arreglo. El desarrollo de clases es una actividad interesante e intelectual fascinante.

El programa de la figura 18.4 demuestra la clase `Array` y sus operadores homónimos. Primero recorramos el programa manejador en `main`. A continuación veamos la definición de clase y cada una de las funciones miembro de la clase y las definiciones de las funciones amigo. El programa produce dos objetos de clase `Array` —`integers1`, con siete elementos, e `integers2`, con un tamaño por omisión de diez elementos (valor por omisión especificado por el constructor de `Array`). El miembro de datos estático `arrayCount` de la clase `Array` contiene el número de objetos `Array` producidos durante la ejecución del programa. La función miembro estática `getArrayCount` devuelve este valor. La función miembro `getSize` devuelve el tamaño del arreglo `integers1`. El programa extrae el tamaño del arreglo `integers1`, y a continuación extrae el arreglo mismo, utilizando el operador de inserción de flujo homónimo, para confirmar que los elementos del arreglo fueron inicializados de forma correcta por el constructor. A continuación, el tamaño del arreglo `integers2`, y después se extrae el arreglo mismo, utilizando el operador de inserción de flujo homónimo.

Al usuario se le pide que introduzca 16 enteros. Para leer dichos valores a ambos arreglos se utiliza el operador de extracción de flujo homónimo, mediante el enunciado

```
cin >> integers1 >> integers2;
```

Los primeros siete valores se almacenan en `integers1` y los restantes en `integers2`. Para confirmar que la entrada fue llevada a cabo correctamente, los dos arreglos son extraídos mediante el operador de inserción de flujo.

A continuación el programa prueba el operador de desigualdad homónimo evaluando la condición.

```
integers1 != integers2
```

y el programa informa que en efecto los arreglos no son iguales.

El programa produce un tercer arreglo, llamado `integers3`, y lo inicializa con el arreglo `integers1`. El programa extrae el tamaño del arreglo `integers3` y a continuación extrae el arreglo mismo, utilizando el operador de inserción de flujo homónimo, para confirmar que los elementos del arreglo fueron inicializados de forma correcta por el constructor.

Después el programa prueba el operador de asignación homónimo (=) con la expresión `integers1=integers2`. Ambos arreglos a continuación son impresos, para confirmar que la asignación fue correcta. Es interesante notar que `integers1` originalmente contenía 7 enteros y para contener una copia de los 10 elementos de `integers2` necesitaba ser redimensionado. Como veremos, el operador de asignación homónimo ejecuta este redimensionamiento de forma transparente para el llamador del operador.

A continuación el programa usa el operador de igualdad homónimo (==) para ver si después de la asignación los objetos `integers1=integers2` son de hecho idénticos.

Entonces el programa utiliza el operador de subíndice homónimo para hacer la referencia al elemento `integers1[5]` un elemento dentro del rango de `integers1`. Este nombre con subíndice se utiliza después como valor a la derecha (*rvalue*) para imprimir el valor en `integers1[5]`, y entonces el nombre se utiliza como valor a la izquierda (*lvalue*) en el lado izquierdo del enunciado de asignación, para recibir un nuevo valor, 1000, correspondiente a `integers1[5]`. Note que el `operator[]` provee la referencia y asigna el valor.

El programa a continuación intenta asignar le valor 1000 a `integers1[15]` un elemento fuera de rango. El programa detecta este error y termina en forma anormal.

De forma interesante, el operador de subíndice de arreglo `[]` no está restringido para uso en arreglos; puede ser utilizado para seleccionar elementos de otros tipos de clases contenedor, como son listas enlazadas, cadenas, diccionarios y demás. También, ya no es necesario que los subíndices sean enteros; se pueden utilizar caracteres o cadenas, por ejemplo.

Ahora que hemos visto cómo funciona este programa, vayamos recorriendo el encabezado de clase y las definiciones de funciones miembro. Las líneas

```
int *ptr;           // pointer to first element of array
int size;           // size of the array
static int arrayCount; // # of Arrays instantiated
```

representa los miembros de datos privados de la clase. El arreglo está formado de un apuntador, `ptr` (al tipo apropiado, en este caso `int`), un miembro `size`, que indica el número de elementos en el arreglo, y un miembro estático `arrayCount`, que indica el número de objetos de arreglo que han sido producidos.

Las líneas

```
friend ostream &operator<<(ostream &, const Array &);
friend istream &operator>>(istream &, Array &);
```

declaran el operador de inserción de flujo homónimo y el operador de extracción de flujo homónimo como amigo de la clase `Array`. Cuando el compilador ve una expresión como

```
cout << arrayObject
```

invoca la función `operator<<` mediante la generación de la llamada

```
operator<<(cout, arrayObject)
```

Cuando el compilador ve una expresión como

```
cin >> arrayObject
```

invoca a la función `operator>>` mediante la generación de la llamada

```
operator>>(cin, arrayObject)
```

Notamos otra vez que esas funciones operador no pueden ser miembros de la clase `Array`, porque el objeto `Array` siempre es mencionado del lado derecho de los operadores de inserción y de extracción de flujo. La función `operator<<` imprime el número de elementos indicados por `size` en el arreglo almacenado en `ptr`. La función `operator>>` introduce directamente al arreglo apuntado por `ptr`. Cada una de estas funciones operador devuelven una referencia apropiada, para permitir llamadas concatenadas.

La línea

```
Array(int arraySize = 10)           // default constructor
```

declara al constructor por omisión correspondiente a la clase y especifica que el tamaño del arreglo se preestablece en 10 elementos. Cuando el compilador ve una declaración como

```
Array integers1(7);
```

o su forma equivalente

```
Array integers1 = 7;
```

invoca al constructor por omisión. La función miembro constructor por omisión `Array` incrementa `arrayCount`, copia el argumento en el miembro de datos `size`, utiliza a `new` para obtener el espacio para contener la representación interna de este arreglo y asigna el apuntador regresado por `new` al miembro de datos `ptr`, utiliza `assert` para probar que `new` tuvo éxito, y a continuación utiliza un ciclo `for` para inicializar a cero todos los elementos del arreglo. Sería perfectamente razonable tener una clase `Array` que no inicializara sus miembros si, por ejemplo, dichos miembros tuvieran que ser leídos en algún momento posterior.

La línea

```
Array(const Array &);           // copy constructor
```

es un *constructor de copia*. Inicializa un objeto `Array` haciendo una copia de un objeto existente `Array`. Dicha copia deberá ser efectuada con cuidado, a fin de evitar el error de que ambos objetos `Array` apunten al mismo almacenamiento asignado dinámicamente, lo que sería exactamente el mismo problema que ocurriría mediante la copia a nivel de miembro por omisión. Los constructores de copia son invocados siempre que se requiera una copia de un objeto, como es tratándose de llamada por valor, al devolver un objeto de una función llamada, o al inicializar un objeto como una copia de otro objeto de la misma clase. En una declaración se llama al constructor de copia cuando un objeto de la clase `Array` es producido e inicializado con otro objeto de la clase `Array`, como ocurre en la declaración siguiente:

```
Array integers3(integers1);
```

o en la declaración equivalente

```
Array integers3 = integers1);
```

La función miembro constructor de copia **Array** incrementa **arrayCount**, copia **size** del arreglo utilizado para la inicialización en el miembro de datos **size**, utiliza **new** para obtener el espacio para contener la representación interna de este arreglo y asigna el apuntador regresado por **new** al miembro de datos **ptr**, utiliza **assert** para probar que **new** tuvo éxito, y después utiliza un ciclo **for** para copiar todos los elementos del arreglo inicializador a este arreglo. Es importante notar que si el constructor de copia simplemente hubiera copiado **ptr** del objeto fuente al objeto destino **ptr**, entonces ambos objetos hubieran señalado al mismo almacenamiento asignado dinámicamente. El primer destructor a ejecutarse hubiera entonces borrado el almacenamiento dinámicamente asignado, y el **ptr** del otro objeto hubiera quedado sin definición, una situación que seguramente causaría un error serio en tiempo de ejecución.

#### *Práctica sana de programación 18.6*

*Un destructor, el operador de asignación, y un constructor de copia para una clase por lo general se proporcionan en grupo.*

La línea

```
~Array(); // destructor
```

declara al destructor de la clase. Cuando se termine la vida de un objeto de la clase **Array**, el destructor será invocado en forma automática. El destructor decrementa **arrayCount** y a continuación utiliza **delete** para recuperar el almacenamiento dinámico creado por **new** en el constructor.

La línea

```
int getSize() const; // return size
```

declara a una función que lee el tamaño del arreglo.

La línea

```
const Array &operator=(const Array &); // assign arrays
```

declara la función operador de asignación homónimo para la clase. Cuando el compilador ve una expresión como

```
integers1 = integers2;
```

invoca a la función **operator=** mediante la generación de la llamada

```
integers1.operator=(integers2)
```

La función miembro **operator=** prueba buscando autoasignación. Si se está intentando hacer una autoasignación, la asignación es pasada por alto (es decir, el objeto ya es él mismo). Si no se trata de una autoasignación, entonces la función miembro utiliza **delete** para recuperar el espacio originalmente asignado en el arreglo destino, copia **size** del arreglo fuente a **size** del arreglo destino, utiliza **new** para asignar dicha cantidad de espacio para el arreglo destino y coloca

el apuntador devuelto por **new** en el miembro **ptr** del arreglo, utiliza **assert** para verificar que **new** tuvo éxito, y después utiliza un ciclo **for** para copiar los elementos del arreglo, correspondientes al arreglo fuente, al arreglo destino. Independiente de que se trate de una autoasignación o no, a continuación la función miembro devuelve el objeto actual (es decir, **\*this**) como referencia constante; esto habilita las asignaciones **Array** concatenadas como **x = y = z**. Si hubiera sido omitida la prueba de autoasignación, la función miembro hubiera empezado borrando el espacio del arreglo destino. Dado que en una autoasignación esto es también el arreglo fuente, el arreglo hubiera sido destruido.

#### *Error común de programación 18.4*

*No probar la existencia y evitar la autoasignación cuando se hace la homonimia del operador de asignación de una clase que incluya un apuntador a almacenamiento dinámico.*

#### *Error común de programación 18.5*

*No proporcionar un operador de asignación homónimo y un constructor de copia para una clase, cuando los objetos de dicha clase contengan apuntadores a almacenamiento dinámicamente asignado.*

#### *Observación de ingeniería de software 18.3*

*Es posible evitar que un objeto de clase sea asignado a otro. Esto se lleva a cabo definiendo el operador de asignación como miembro privado de la clase.*

La línea

```
int operator==(const Array &) const; // compare equal
```

declara al operador de igualdad homónimo (**==**) para la clase. Cuando el compilador ve la expresión

```
integers1 == integers2
```

en **main**, invoca la función miembro **operator==** mediante la generación de la llamada

```
integers1.operator==(integers2)
```

La función miembro **operator==** de inmediato devuelve 0 (falso) si son diferentes los miembros **size** de los arreglos. De lo contrario, la función miembro compara cada par de elementos. Si todos son iguales, devuelve 1 (verdadero). El primer par de elementos que sea distinto hará que se devuelva de inmediato 0.

La línea

```
int operator!=(const Array &) const; // compare !equal
```

declara el operador de desigualdad homónimo (**!=**) para la clase. Invocará a la función miembro **operator!=** y operará en forma similar a la función de operador de igualdad homónimo.

La línea

```
int &operator[] (int); // subscript operator
```

declara al operador de subíndice homónimo correspondiente a la clase. Cuando el compilador ve la expresión

```
integers1[5]
```

en **main**, el compilador invoca la función miembro homónima **operator[]** generando la llamada

```
integers1.operator[](5)
```

La función miembro **operator[]** prueba si el subíndice está dentro de rango, y si no es así, el programa termina en forma anormal. Si el subíndice está dentro de rango, devuelve el elemento apropiado del arreglo como referencia, de manera que pueda ser utilizado como valor a la izquierda (*lvalue*) (por ejemplo, en el lado izquierdo de un enunciado de asignación).

La línea

```
static int getArrayCount();           // return count of Arrays
```

declara la función miembro estática **getArrayCount** que devuelve el valor del miembro de datos estático **arrayCount**, aún si no existen objetos de la clase **Array**.

```
// ARRAY1.H
// Simple class Array (for integers)
#ifndef ARRAY1_H
#define ARRAY1_H

#include <iostream.h>

class Array {
    friend ostream &operator<<(ostream &, const Array &);
    friend istream &operator>>(istream &, Array &);
public:
    Array(int = 10);           // default constructor
    Array(const Array &);      // copy constructor
    ~Array();                  // destructor
    int getSize() const;       // return size
    const Array &operator=(const Array &); // assign arrays
    int operator==(const Array &) const; // compare equal
    int operator!=(const Array &) const; // compare !equal
    int &operator[](int);      // subscript operator
    static int getArrayCount(); // return count of
                                // arrays instantiated
private:
    int *ptr; // pointer to first element of array
    int size; // size of the array
    static int arrayCount; // # of Arrays instantiated
};

#endif
```

Fig. 18.4 Definición de la clase **Array** (parte 1 de 7).

```
// ARRAY1.CPP
// Member function definitions for class Array
#include <iostream.h>
#include <stdlib.h>
#include <assert.h>
#include "array1.h"

// Initialize static data member at file scope
int Array::arrayCount = 0; // no objects yet

// Return the number of Array objects instantiated
int Array::getArrayCount() { return arrayCount; }

// Default constructor for class Array
Array::Array(int arraySize)
{
    ++arrayCount;           // count one more object
    size = arraySize;       // default size is 10
    ptr = new int[size];    // create space for array
    assert(ptr != 0);       // terminate if memory not allocated

    for (int i = 0; i < size; i++)
        ptr[i] = 0;        // initialize array
}

// Copy constructor for class Array
Array::Array(const Array &init)
{
    ++arrayCount;           // count one more object
    size = init.size;       // size this object
    ptr = new int[size];    // create space for array
    assert(ptr != 0);       // terminate if memory not allocated

    for (int i = 0; i < size; i++)
        ptr[i] = init.ptr[i]; // copy init into object
}

// Destructor for class Array
Array::~Array()
{
    --arrayCount;           // one fewer objects
    delete [] ptr;          // reclaim space for array
}

// Get the size of the array
int Array::getSize() const { return size; }

// Overloaded subscript operator
int &Array::operator[](int subscript)
{
    // check for subscript out of range error
    assert(0 <= subscript && subscript < size);

    return ptr[subscript]; // reference return creates lvalue
}
```

Fig. 18.4 Definiciones de funciones miembro para la clase **Array** (parte 2 de 7).

```

// Determine if two arrays are equal and
// return 1 if true, 0 if false.
int Array::operator==(const Array &right) const
{
    if (size != right.size)
        return 0;    // arrays of different sizes

    for (int i = 0; i < size; i++)
        if (ptr[i] != right.ptr[i])
            return 0; // arrays are not equal

    return 1;    // arrays are equal
}

// Determine if two arrays are not equal and
// return 1 if true, 0 if false.
int Array::operator!=(const Array &right) const
{
    if (size != right.size)
        return 1;    // arrays of different sizes

    for (int i = 0; i < size; i++)
        if (ptr[i] != right.ptr[i])
            return 1;    // arrays are not equal

    return 0;    // arrays are equal
}

// Overloaded assignment operator
const Array &Array::operator=(const Array &right)
{
    if (&right != this) {    // check for self-assignment
        delete [] ptr;    // reclaim space
        size = right.size;    // resize this object
        ptr = new int[size];    // create space for array copy
        assert(ptr != 0);    // terminate if memory not allocated

        for (int i = 0; i < size; i++)
            ptr[i] = right.ptr[i];    // copy array into object
    }

    return *this;    // enables x = y = z;
}

// Overloaded input operator for class Array;
// inputs values for entire array.
istream &operator>>(istream &input, Array &a)
{
    for (int i = 0; i < a.size; i++)
        input >> a.ptr[i];

    return input;    // enables cin >> x >> y;
}

```

Fig. 18.4 Definiciones de funciones miembro para la clase **Array** (parte 3 de 7).

```

// Overloaded output operator for class Array
ostream &operator<<(ostream &output, const Array &a)
{
    for (int i = 0; i < a.size; i++) {
        output << a.ptr[i] << ' ';

        if ((i + 1) % 10 == 0)
            output << endl;
    }

    if (i % 10 != 0)
        output << endl;

    return output;    // enables cout << x << y;
}

```

Fig.18.4 Definiciones de función miembro para la clase **Array** (parte 4 de 7).

```

// FIG18_4.CPP
// Driver for simple class Array
#include <iostream.h>
#include "array1.h"

main()
{
    // no objects yet
    cout << "# of arrays instantiated = "
         << Array::getArrayCount() << '\n';

    // create two arrays and print Array count
    Array integers1(7), integers2;
    cout << "# of arrays instantiated = "
         << Array::getArrayCount();

    // print integers1 size and contents
    cout << "\n\nSize of array integers1 is "
         << integers1.getSize()
         << "\nArray after initialization:\n"
         << integers1;

    // print integers2 size and contents
    cout << "\nSize of array integers2 is "
         << integers2.getSize()
         << "\nArray after initialization:\n"
         << integers2;

    // input and print integers1 and integers2
    cout << "\nInput 17 integers:\n";
    cin >> integers1 >> integers2;
    cout << "After input, the arrays contain:\n"
         << "integers1: " << integers1
         << "integers2: " << integers2;
}

```

Fig. 18.4 Manejador para la clase **Array** (parte 5 de 7).

```

// create array integers3 using integers1 as an
// initializer; print size and contents
Array integers3(integers1);

cout << "\nSize of array integers3 is "
    << integers3.getSize()
    << "\nArray after initialization:\n"
    << integers3;

// use overloaded inequality (!=) operator
cout << "\nEvaluating: integers1 != integers2\n";
if (integers1 != integers2)
    cout << "They are not equal\n\n";

// use overloaded assignment (=) operator
cout << "Assigning integers2 to integers1:\n";
integers1 = integers2;
cout << "integers1: " << integers1
    << "integers2: " << integers2;

// use overloaded equality (==) operator
cout << "\nEvaluating: integers1 == integers2\n";
if (integers1 == integers2)
    cout << "They are equal\n\n";

// use overloaded subscript operator to create rvalue
cout << "integers1[5] is " << integers1[5] << endl;

// use overloaded subscript operator to create lvalue
cout << "Assigning 1000 to integers1[5]\n";
integers1[5] = 1000;
cout << "integers1: " << integers1;

// attempt to use out of range subscript
cout << "\nAttempt to assign 1000 to integers1[15]\n";
integers1[15] = 1000; // ERROR: out of range

return 0;
}

```

Fig. 18.4 Manejador para la clase **Array** (parte 6 de 7).

## 18.9 Conversión entre tipos

La mayor parte de los programas procesan información en una variedad de tipos. Algunas veces todas las operaciones “se mantienen dentro de un tipo”. Por ejemplo, añadir un entero a un entero produce un entero (siempre y cuando el resultando no sea tan grande que no pueda ser representado como un entero). Pero, a menudo es necesario convertir datos de un tipo a datos de otro tipo. Esto puede ocurrir en asignaciones, en cálculos, al pasar valores a funciones, y al devolver valores de funciones. El compilador sabe cómo llevar a cabo conversiones entre tipos incorporados. Los programadores pueden obligar a conversiones entre tipos incorporados mediante las conversiones explícitas *casting*.

```

# of arrays instantiated = 0
# of arrays instantiated = 2

Size of array integers1 is 7
Array after initialization:
0 0 0 0 0 0 0

Size of array integers2 is 10
Array after initialization:
0 0 0 0 0 0 0 0 0 0

Input 17 integers:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
After input, the arrays contain:
integers1: 1 2 3 4 5 6 7
integers2: 8 9 10 11 12 13 14 15 16 17

Size of array integers3 is 7
Array after initialization:
1 2 3 4 5 6 7

Evaluating: integers1 != integers2
They are not equal

Assigning integers2 to integers1:
integers1: 8 9 10 11 12 13 14 15 16 17
integers2: 8 9 10 11 12 13 14 15 16 17

Evaluating: integers1 == integers2
They are equal

integers1[5] is 13
Assigning 1000 to integers1[5]
integers1: 8 9 10 11 12 13 14 15 16 17

Attempt to assign 1000 to integers1[15]
Assertion failed: 0 <= subscript && subscript < size,
file ARRAY1.CPP, line 98
Abnormal program termination

```

Fig. 18.4 Salida del manejador para la clase **Array** (parte 7 de 7).

Pero ¿qué pasa en relación con los tipos definidos por usuario? El compilador no puede saber en forma automática cómo convertir entre tipos definidos por usuario y tipos incorporados. El programador deberá especificar en forma explícita cómo deberán ocurrir dichas conversiones. Estas conversiones pueden ser ejecutadas mediante *constructores de conversión*, es decir, constructores de un solo argumento, que sólo conviertan objetos de otros tipos en objetos de una clase particular. Más adelante en este capítulo utilizaremos un constructor de conversión para convertir las cadenas **char \*** normales en objetos **String** de C++.

Un *operador de conversión* (también conocido como *operador de conversión explícita cast*) puede ser utilizado para convertir un objeto de una clase en un objeto de otra clase o en un objeto

de un tipo incorporado. Este operador de conversión debe ser una función miembro no estática; este tipo de operador de conversión no puede ser una función amigo.

El prototipo de función

```
operator char *() const;
```

declara una función operador de conversión explícita `cast` homónima para crear un objeto temporal `char*` partiendo de un objeto de un tipo definido por usuario. Una función operador de conversión explícita `cast` homónima no especifica un tipo de regreso —el tipo de regreso es el tipo al cual se está convirtiendo el objeto. Si `s` es un objeto de clase, cuando el compilador vea la expresión `(char *) s` el compilador generará la llamada `s.operator char *()`. El operando `s` es el objeto de clase `s` para el cual se invoca la función miembro `operator char *`.

Las funciones operador de conversión explícita `cast` homónimas pueden ser definidas para la conversión de objetos de tipos definidos por usuario a tipos incorporados o a objetos de otros tipos definidos por usuario. Los prototipos de función

```
operator int() const;
operator otherClass() const;
```

declaran funciones operador de conversión explícita `cast` homónimas para la conversión de un objeto de un tipo definido por usuario a un entero o para convertir un objeto de un tipo definido por usuario a un objeto de un tipo definido por usuario `otherClass`.

Una de las características maravillosas de los operadores de conversión explícita (`cast`) y de los constructores de conversión es que, cuando es necesario, el compilador puede llamar estas funciones en forma automática para crear objetos temporales. Por ejemplo, si en un programa aparece un objeto `s` de una clase `String` definida por usuario, en una posición donde se espera un `char *` normal, tal como

```
cout << s;
```

el compilador llama la función de operador de conversión explícita (`cast`) homónima `operator char *` para convertir el objeto en un `char *` y utiliza el `char *` resultante dentro de la expresión.

## 18.10 Estudio de caso: una clase `String`

Como un ejercicio de recapitulación a nuestro estudio de la homonimia construiremos una clase que maneje la creación y manipulación de cadenas. Ni C ni C++ tienen un tipo de datos de cadenas incorporado. Pero C++ nos permite añadir como clase a nuestro propio tipo de cadenas, y —mediante los mecanismos de la homonimia— proporciona un conjunto de operadores para manejar convenientemente las cadenas. Las varias partes de la figura 18.5 incluyen un encabezado de clase, definiciones de funciones miembros, y un programa manejador para probar nuestra nueva clase `String`.

Primero, presentaremos el encabezado para la clase `String`. Analizamos los datos privados utilizados para representar objetos `String`. A continuación, recorremos la interfaz pública de la clase, analizando cada uno de los servicios que la misma proporciona.

A continuación, recorreremos el programa manejador en `main`. Analizaremos el estilo de codificación al cual “aspiramos”, es decir, los tipos de expresiones operador intensivo que nos gustaría tener la capacidad de escribir, con objetos de nuestra nueva clase `String` y con el conjunto de operadores homónimos de la clase.

A continuación, analizamos las definiciones de funciones miembro correspondientes a la clase `String`. Para cada uno de los operadores homónimos, mostramos el código en el programa manejador que invoca a la función operador homónima, y explicamos cómo funciona la función operador homónima.

Empezamos con la representación interna de un `String`. Las líneas

```
private:
    char *sPtr;           // pointer to start of string
    int length;           // string length
```

declaran los miembros de datos privados de la clase. Un objeto `String` tiene un apuntador a su almacenamiento dinámico asignado representando la cadena de caracteres, y tiene un campo de longitud que representa el número de caracteres en la cadena, excluyendo el carácter nulo al final de la cadena de caracteres.

Ahora recorramos el archivo de cabecera de la clase `String` de la figura 18.5. Las líneas

```
friend ostream &operator<<(ostream &, const String &);
friend istream &operator>>(istream &, String &);
```

declaran la función operador de inserción de flujo homónima `operator<<` y la función operador de extracción de flujo homónima `operator>>` como amigo de la clase. La puesta en práctica de las funciones anteriores es simple.

La línea

```
String(const char * = ""); // conversion constructor
```

declara un *constructor de conversión*. Este constructor toma un argumento `char *` (que por omisión es la cadena vacía) y produce un objeto `String` que incluye la misma cadena de caracteres. Cualquier constructor de un argumento puede ser considerado como si fuera un constructor de conversión. Como veremos, estos constructores son útiles cuando estamos haciendo asignaciones de cadenas de caracteres a objetos `String`. El constructor de conversión convierte la cadena convencional en un objeto `String`, que a continuación es asignado al objeto destino `String`. La disponibilidad de este constructor de conversión significa que para asignar en forma específica las cadenas de caracteres a los objetos `String` no es necesario proveer un operador de asignación homónimo. El compilador invoca en forma automática el constructor de conversión, a fin de crear un objeto temporal `String` que contenga la cadena de caracteres. Entonces, se invoca el operador de asignación homónimo para asignar el objeto temporal `String` a otro objeto `String`. Note que cuando C++ utiliza de esta forma un constructor de conversión, sólo puede aplicar un constructor para intentar hacer coincidir las necesidades del operador de asignación homónimo. No es posible hacer coincidir las necesidades del operador homónimo llevando a cabo una serie de conversiones implícitas, definidas por el usuario. El constructor de conversión `String` podría ser invocado en una declaración como `String s1 ("happy")`. La función miembro constructor de conversión calcula la longitud de la cadena de caracteres y asigna este cálculo al miembro de datos privado `length`, utiliza `new` para asignar suficiente espacio al miembro de datos privado `sPtr`, utiliza `assert` para probar que `new` tuvo éxito, y si así fue, utiliza `strcpy` para copiar la cadena de caracteres al objeto.

La línea

```
String(const String &); // copy constructor
```

es un constructor de copia. Inicializa un objeto **String** haciendo una copia de un objeto existente **String**. Esta copia deberá ser efectuada con cuidado para evitar el problema de tener ambos objetos **String** señalando al mismo almacenamiento asignado dinámicamente, exactamente el problema que ocurriría tratándose de una copia a nivel de miembro por omisión. El constructor de copia opera de forma similar al constructor de conversión, excepto que simplemente copia el miembro **length** del objeto fuente **String** al objeto destino **String**. Note que el constructor de copia genera un nuevo espacio para la cadena de caracteres interna del objeto destino. Si solo copiara **sPtr** en el objeto fuente al **sPtr** del objeto destino, entonces ambos objetos señalarían al mismo almacenamiento dinámico asignado. El primer destructor que se ejecutase borraría entonces el almacenamiento dinámicamente asignado y el **sPtr** del otro objeto quedaría entonces sin definir, una situación que probablemente causaría un error serio en tiempo de ejecución.

La línea

```
~String(); // destructor
```

declara al destructor correspondiente a la clase **String**. El destructor utiliza **delete** para reclamar el almacenamiento dinámico obtenido por **new** en los constructores para proveer el espacio correspondiente a la cadena de caracteres.

La línea

```
const String &operator=(const String &); // assignment
```

declara al operador de asignación homónimo. Cuando el compilador ve una expresión como **string1 = string2**, genera la llamada de función

```
string1.operator=(string2);
```

La función operador de asignación homónimo **operator=** prueba por si existe autoasignación, exactamente de la misma forma que lo hizo el constructor de copia. Si se trata de una autoasignación, la función sólo regresa, dado que el objeto ya es él mismo. Si esta prueba se omitiera, la función de inmediato borraría el espacio en el objeto destino y se perdería la cadena de caracteres. Suponiendo que no se trata de una autoasignación, la función no borra el espacio, copia el campo de longitud del objeto fuente al objeto destino, genera un nuevo espacio para el objeto destino, utiliza **assert** para determinar si **new** tuvo éxito, y entonces utiliza **strcpy** para copiar la cadena de caracteres del objeto fuente al objeto destino. Independiente de que se trate o no de una autoasignación, se devuelve **\* this** para permitir asignaciones concatenadas.

La línea

```
String &operator+=(const String &); // concatenation
```

declara al operador de concatenación de cadenas homónimo. Cuando el compilador ve la expresión **s1 += s2** en **main**, genera la llamada de función **s1.operator+=(s2)**. La función **operator+=** crea un apuntador temporal para contener el apuntador de cadena de caracteres del objeto actual, en tanto pueda ser borrada la memoria de la cadena de caracteres, calcula la longitud combinada de la cadena concatenada, utiliza **new** para reservar espacio para la cadena, utiliza **assert** para probar si **new** tuvo éxito, utiliza **strcpy** para copiar la cadena original al nuevo espacio asignado, utiliza **strcat** para concatenar la cadena de caracteres del objeto fuente al nuevo espacio asignado, utiliza **delete** para recuperar el espacio ocupado por la cadena de

caracteres original de este objeto, y devuelve **\* this** como un **String &** para permitir la concatenación de los operadores **+=**.

¿Será necesario tener un segundo operador de concatenación homónimo para poder hacer la concatenación de un **String** y de un **char \***? No, el constructor de conversión **const char \*** convierte una cadena convencional en un objeto **String**, que a continuación coincide con el operador de concatenación homónimo. C++ puede llevar a cabo estas conversiones en un nivel de profundidad, para facilitar una coincidencia. C++ también puede, antes de llevar a cabo la conversión entre un tipo incorporado y una clase, ejecutar una conversión implícita definida por compilador entre tipos incorporados.

### Sugerencia de rendimiento 18.2

Usar el operador de concatenación homónimo **+=** que toma un solo argumento del tipo **const char \*** da resultados más eficaces que primero tener que efectuar la conversión implícita y a continuación, la concatenación. Las conversiones implícitas requieren de menos código y causan menos errores.

La línea

```
int operator!() const; // is String empty?
```

declara el operador de negación homónimo. Este operador se usa por lo común con las clases de cadenas, para probar si una cadena está vacía. Por ejemplo, cuando el compilador ve la expresión **!string1**, genera la llamada de función

```
string1.operator!()
```

Esta función sencillamente devuelve el resultado de probar si **length** es igual a cero.

Las líneas

```
int operator==(const String &) const; // test s1 == s2
int operator!=(const String &) const; // test s1 != s2
int operator<(const String &) const; // test s1 < s2
int operator>(const String &) const; // test s1 > s2
int operator>=(const String &) const; // test s1 >= s2
int operator<=(const String &) const; // test s1 <= s2
```

declaran los operadores de igualdad homónimos y los operadores relacionales homónimos, correspondientes a la clase **String**. Todos ellos son similares, por lo que analizaremos un ejemplo, simplemente la homonimia del operador **>=**. Cuando el compilador ve la expresión **string1 >= string2**, generará la llamada de función

```
string1.operator>=(string2)
```

misma que devolverá 1 (verdadero) si **string1** es mayor que o igual a **string2**. Cada uno de estos operadores utiliza **strcmp** para comparar las cadenas de caracteres en los objetos **String**.

La línea

```
char &operator[](int); // return char reference
```

declara el operador de subíndice homónimo. Cuando el compilador ve una expresión como **string1[0]**, generará la llamada de función **string1.operator[](0)**. La función **ope-**

**rator []** primero utiliza **assert** para llevar a cabo una verificación de rango sobre el subíndice; si el subíndice está fuera de rango, el programa imprimirá un mensaje de error y terminará en forma anormal. Si el subíndice está dentro de rango, el operador **operator []** devuelve el carácter apropiado como un **char &** del objeto **String**; este **char &** puede ser utilizado como un valor a la izquierda (*lvalue*) para modificar el carácter designado del objeto **String**.

La línea

```
String &operator() (int, int);    // return a substring
```

declara el *operador de llamada de función homónimo*. En las clases de cadenas, es común hacer la homonimia de este operador para seleccionar una subcadena de un objeto **String**. Los dos parámetros enteros especifican la posición inicial y la longitud de la subcadena que se está seleccionando de **String**. Si la posición inicial está fuera de rango o si es negativa la longitud de la subcadena, se generará un mensaje de error. Por regla convencional, si la longitud de la subcadena es 0, entonces la subcadena será seleccionada hasta el final del objeto **String**. Por

```
// STRING2.H
// Definition of a String class
#ifndef STRING1_H
#define STRING1_H

#include <iostream.h>

class String {
    friend ostream &operator<<(ostream &, const String &);
    friend istream &operator>>(istream &, String &);
public:
    String(const char * = ""); // conversion constructor
    String(const String &);    // copy constructor
    ~String();                 // destructor
    const String &operator=(const String &); // assignment
    String &operator+=(const String &);    // concatenation
    int operator!() const;                 // is String empty?
    int operator==(const String &) const; // test s1 == s2
    int operator!=(const String &) const; // test s1 != s2
    int operator<(const String &) const;  // test s1 < s2
    int operator>(const String &) const;  // test s1 > s2
    int operator>=(const String &) const; // test s1 >= s2
    int operator<=(const String &) const; // test s1 <= s2
    char &operator[](int);                // return char reference
    String &operator()(int, int);         // return a substring
    int getLength() const;               // return string length
private:
    char *sPtr;                          // pointer to start of string
    int length;                          // string length
};

#endif
```

Fig. 18.5 Definición de una clase **String** básica (parte 1 de 8).

ejemplo, suponga que **string1** es un objeto **String** que contiene la cadena de caracteres "AEIOU". Cuando el compilador ve la expresión **string1(2, 2)**, genera la llamada de función **string1.operator() (2, 2)**. Cuando esta llamada se ejecuta, se produce un objeto temporal **String**, que contiene la cadena "IO" y devuelve una referencia a este objeto. Hacer la homonimia del operador de llamada de función **()** resulta poderosa porque las funciones pueden tomar listas de parámetros arbitrariamente largas y complejas. Por lo tanto, podemos utilizar esta capacidad para muchos fines interesantes. Otro uso del operador de llamada de función es una notación alterna de subscripción de los arreglos: en vez de utilizar la notación de dobles corchetes de C, que resultan torpes tratándose de dobles arreglos, como en **a[b] [c]**, por ejemplo, algunos programadores prefieren hacer la homonimia del operador de llamada de función para permitir la notación **a(b, c)**. El operador de llamada de función homónimo solamente puede ser una función miembro no estática. Este operador se utiliza sólo cuando "el nombre de función" es un objeto de la clase **String**.

La línea

```
int getLength() const;           // return string length
```

declara una función que devuelve la longitud del objeto **String**. Note que esta función obtiene la longitud, regresando el valor de los datos privados de la clase **String**.

Llegado a este punto, el lector deberá recorrer el código en **main**, examinar la ventana de salida, y verificar el uso de cada operador homónimo.

```
// STRING2.CPP
// Member function definitions for class String

#include <iostream.h>
#include <string.h>
#include <assert.h>
#include "string2.h"

// Conversion constructor: Convert char * to String
String::String(const char *s)
{
    cout << "Conversion constructor: " << s << endl;
    length = strlen(s); // compute length
    sPtr = new char[length + 1]; // allocate storage
    assert(sPtr != 0); // terminate if memory not allocated
    strcpy(sPtr, s); // copy literal to object
}

// Copy constructor
String::String(const String &copy)
{
    cout << "Copy constructor: " << copy.sPtr << endl;
    length = copy.length; // copy length
    sPtr = new char[length + 1]; // allocate storage
    assert(sPtr != 0); // ensure memory allocated
    strcpy(sPtr, copy.sPtr); // copy string
}
```

Fig. 18.5 Definiciones de funciones miembro para la clase **String** (parte 2 de 8).

```

// Destructor
String::~String()
{
    cout << "Destructor: " << sPtr << endl;
    delete [] sPtr;          // reclaim string
}

// Overloaded = operator; avoids self assignment
const String &String::operator=(const String &right)
{
    cout << "operator= called" << endl;

    if (&right != this) {      // avoid self assignment
        delete [] sPtr;        // prevents memory leak
        length = right.length; // new String length
        sPtr = new char[length + 1]; // allocate memory
        assert(sPtr != 0);      // ensure memory allocated
        strcpy(sPtr, right.sPtr); // copy string
    }
    else
        cout << "Attempted assignment of a String to itself\n";

    return *this; // enables concatenated assignments
}

// Concatenate right operand to this object and
// store in this object.
String &String::operator+=(const String &right)
{
    char *tempPtr = sPtr;      // hold to be able to delete
    length += right.length;     // new String length
    sPtr = new char[length + 1]; // create space
    assert(sPtr != 0); // terminate if memory not allocated
    strcpy(sPtr, tempPtr);      // left part of new String
    strcat(sPtr, right.sPtr);   // right part of new String
    delete [] tempPtr;          // reclaim old space
    return *this;               // enables concatenated calls
}

// Is this String empty?
int String::operator!() const { return length == 0; }

// Is this String equal to right String?
int String::operator==(const String &right) const
{ return strcmp(sPtr, right.sPtr) == 0; }

// Is this String not equal to right String?
int String::operator!=(const String &right) const
{ return strcmp(sPtr, right.sPtr) != 0; }

// Is this String less than right String?
int String::operator<(const String &right) const
{ return strcmp(sPtr, right.sPtr) < 0; }

```

Fig. 18.5 Definiciones de funciones miembro para la clase **String** (parte 3 de 8).

```

// Is this String greater than right String?
int String::operator>(const String &right) const
{ return strcmp(sPtr, right.sPtr) > 0; }

// Is this String greater than or equal to right String?
int String::operator>=(const String &right) const
{ return strcmp(sPtr, right.sPtr) >= 0; }

// Is this String less than or equal to right String?
int String::operator<=(const String &right) const
{ return strcmp(sPtr, right.sPtr) <= 0; }

// Return a reference to a character in a String.
char &String::operator[](int subscript)
{
    // First test for subscript out of range
    assert(subscript >= 0 && subscript < length);

    return sPtr[subscript]; // creates lvalue
}

// Return a substring beginning at index and
// of length subLength as a reference to a String object.
String &String::operator()(int index, int subLength)
{
    // ensure index is in range and substring length >= 0
    assert(index >= 0 && index < length && subLength >= 0);

    String *subPtr = new String; // empty String
    assert(subPtr != 0); // ensure new String allocated

    // determine length of substring
    int size;

    if ((subLength == 0) || (index + subLength > length))
        size = length - index + 1;
    else
        size = subLength + 1;

    // allocate memory for substring
    delete subPtr->sPtr;
    subPtr->length = size;
    subPtr->sPtr = new char[size];
    assert(subPtr->sPtr != 0); // ensure space allocated

    // copy substring into new String
    for (int i = index, j = 0; i < index + size - 1; i++, j++)
        subPtr->sPtr[j] = sPtr[i];

    subPtr->sPtr[j] = '\0'; // terminate the new String
    return *subPtr;        // return new String
}

```

Fig. 18.5 Definiciones de funciones miembro para la clase **String** (parte 4 de 8).

```

// Return string length
int String::getLength() const { return length; }

// Overloaded output operator
ostream &operator<<(ostream &output, const String &s)
{
    output << s.sPtr;
    return output;    // enables concatenation
}

// Overloaded input operator
istream &operator>>(istream &input, String &s)
{
    static char temp[100];    // buffer to store input

    input >> temp;
    s = temp;    // use String class assignment operator
    return input;    // enables concatenation
}

```

Fig. 18.5 Definiciones de funciones miembro para la clase String (parte 5 de 8).

```

// FIG18_5.CPP
// Driver for class String
#include <iostream.h>
#include "string2.h"

main()
{
    String s1("happy"), s2(" birthday"), s3;

    // test overloaded equality and relational operators
    cout << "s1 is \"" << s1 << "\"; s2 is \"" << s2
        << "\"; s3 is empty\n"
        << "The results of comparing s2 and s1:"
        << "\ns2 == s1 yields " << (s2 == s1)
        << "\ns2 != s1 yields " << (s2 != s1)
        << "\ns2 > s1 yields " << (s2 > s1)
        << "\ns2 < s1 yields " << (s2 < s1)
        << "\ns2 >= s1 yields " << (s2 >= s1)
        << "\ns2 <= s1 yields " << (s2 <= s1) << "\n\n";

    // test overloaded String empty (!) operator
    cout << "Testing !s3:\n";
    if (!s3) {
        cout << "s3 is empty; assigning s1 to s3;\n";
        s3 = s1;    // test overloaded assignment
        cout << "s3 is \"" << s3 << "\"\n\n";
    }
}

```

Fig. 18.5 Manejador para probar la clase String (parte 6 de 8).

```

// test overloaded String concatenation operator
cout << "s1 += s2 yields s1 = ";
s1 += s2;    // test overloaded concatenation
cout << s1 << "\n\n";

// test conversion constructor
cout << "s1 += \" to you\" yields\n";
s1 += " to you";    // test conversion constructor
cout << "s1 = " << s1 << "\n\n";

// test overloaded function call operator () for substring
cout << "The substring of s1 starting at\n"
    << "location 0 for 14 characters, s1(0, 14), is: "
    << s1(0, 14) << "\n\n";

// test substring "to-end-of-String" option
cout << "The substring of s1 starting at\n"
    << "location 15, s1(15, 0), is: "
    << s1(15, 0) << "\n\n";    // 0 means "to end of string"

// test copy constructor
String *s4Ptr = new String(s1);
cout << "**s4Ptr = " << *s4Ptr << "\n\n";

// test assignment (=) operator with self-assignment
cout << "assigning *s4Ptr to *s4Ptr\n";
*s4Ptr = *s4Ptr;    // test overloaded self-assignment
cout << "**s4Ptr = " << *s4Ptr << endl;

// test destructor
delete s4Ptr;

// test using subscript operator to create lvalue
s1[0] = 'H';
s1[6] = 'B';
cout << "\ns1 after s1[0] = 'H' and s1[6] = 'B' is: "
    << s1 << "\n\n";

// test subscript out of range
cout << "Attempt to assign 'd' to s1[30] yields:\n";
s1[30] = 'd';    // ERROR: subscript out of range

return 0;
}

```

Fig. 18.5 Manejador para probar la clase String (parte 7 de 8).

## 18.11 Homonimia de ++ y --

Todos los operadores de incremento y de decremento —preincremento, postincremento, predecremento y postdecremento pueden tener homónimos. Veremos cómo el compilador puede conocer cuál es la versión prefija o postfija del operador de incremento o de decremento que está siendo utilizada.

```

Conversion constructor: happy
Conversion constructor: birthday
Conversion constructor:
s1 is "happy"; s2 is "birthday"; s3 is empty
The results of comparing s2 and s1:
s2 == s1 yields 0
s2 != s1 yields 1
s2 > s1 yields 0
s2 < s1 yields 1
s2 >= s1 yields 0
s2 <= s1 yields 1

Testing !s3:
s3 is empty; assigning s1 to s3;
operator= called
s3 is "happy"

s1 += s2 yields s1 = happy birthday

s1 += "to you" yields
Conversion constructor: to you
Destructor: to you
s1 = happy birthday to you

Conversion constructor:
The substring of s1 starting at
location 0 for 14 characters, s1(0, 14), is: happy birthday

Conversion constructor:
The substring of s1 starting at
location 15, s1(15, 0), is: to you

Copy constructor: happy birthday to you
*s4Ptr = happy birthday to you

Assigning *s4Ptr to *s4Ptr
operator= called
Attempted assignment of a String to itself
*s4Ptr = happy birthday to you
Destructor: happy birthday to you

s1 after s1[0] = 'H' and s1[6] = 'B' is: Happy Birthday to you

Attempt to assign 'd' to s1[30] yields:
Assertion failed: subscript >= 0 && subscript < length,
file STRING2.CPP, line 98
Abnormal program termination

```

Fig. 18.5 Salida del manejador para probar la clase `String` (parte 8 de 8).

Para hacer la homonimia del operador incremental para permitir el uso tanto de preincremento como de postincremento, cada función de operador homónimo debe tener una firma distinta, de tal forma que el compilador sea capaz de determinar cuál es la versión de `++` que se desea. Se hace la homonimia de las versiones prefijas de la misma manera que cualquier operador unario prefijo.

Suponga, por ejemplo, que en el objeto `d1` de `Date` deseamos añadir 1 al día. Cuando el compilador ve la expresión preincremental

```
++d1
```

el compilador genera la llamada de función miembro

```
d1.operator++()
```

cuyo prototipo sería

```
Date operator++();
```

Si el preincremento es puesto en práctica como una función no miembro, cuando el compilador vea la expresión

```
++d1
```

el compilador generará la llamada de función

```
operator++(d1)
```

cuyo prototipo debería ser declarado en la clase `Date` como

```
friend Date operator++(Date &);
```

Hacer la homonimia del operador de postincremento resulta un pequeño desafío, porque el compilador debe ser capaz de distinguir las firmas de las funciones de operador de preincremento y postincremento homónimas. La regla convencional que ha sido adoptada en C++, es que cuando el compilador vea la expresión de postincremento

```
d1++
```

generará la llamada de función miembro

```
d1.operator++(0)
```

cuyo prototipo sería

```
Date operator++(int)
```

Aquí el 0 es un valor estricto de relleno, con el propósito que sea distinguible la lista de argumentos de `operator++` utilizada para postincrementar, de la lista de argumentos de `operator++` utilizada para preincrementar.

Si el postincremento es puesto en práctica como función no miembro, cuando el compilador vea la expresión

```
d1++
```

el compilador generará la llamada de función

```
operator++(d1, 0)
```

cuyo prototipo sería

```
friend Date operator++(Date &, int);
```

Otra vez, el argumento `0 int` es utilizado por el compilador para que la lista de argumentos de `operator++` utilizado para postincrementar, resulte distinguible de la lista de argumentos para preincrementar. Todo lo que hemos dicho en esta sección para la homonimia de operadores de preincremento y postincremento es aplicable a la homonimia de los operadores de predecremento y posdecremento. En la sección siguiente, examinaremos una clase `Date`, que utiliza operadores de preincremento y postincremento homónimos.

## 18.12 Estudio de caso: una clase `Date`

En la figura 18.6 se ilustra una clase `Date`. La clase utiliza operadores de preincremento y de postincremento homónimos, para añadir 1 al día en un objeto `Date`.

La clase proporciona las siguientes funciones miembro: un operador de inserción de flujo homónimo, un constructor por omisión, una función `setDate`, una función operador de preincremento homónima, una función operador de postincremento homónima, y un operador de asignación de adición homónimo (`+=`).

El programa manejador en `main` crea los objetos de fecha `d1` que se inicializan a January 1, 1990; `d2`, que se inicializan a December 27, 1992; y `d3` que se inicializan a una fecha inválida. El constructor `Date` llama a `setDate` para validar el mes, día y año específico. Si el mes es inválido, se define como 1. Un año inválido se define como 1900. Un día inválido se define como 1.

El programa manejador extrae cada uno de los objetos construidos `Date` utilizando el operador de inserción de flujo homónimo. El operador homónimo `+=` es utilizado para añadir 7 días a `d2`. A continuación la función `setDate` es usada para definir `d3` a February 28, 1992. Después, un nuevo objeto `Date`, `d4`, se define a March 18, 1969. Entonces mediante el operador de preincremento homónimo, `d4` se incrementa en 1. La fecha es impresa, antes y después del preincremento, para confirmar que se hizo correctamente. Por último, `d4` es incrementado utilizando el operador de postincremento homónimo. La fecha es impresa antes y después del postincremento, a fin de confirmar que funcionó correctamente.

La homonimia del operador de preincremento es directa. Para efectuar el incremento real el programa llama a la función de utilidad `helpIncrement`. Esta función debe ocuparse de los desbordamientos que ocurren cuando intentamos aumentarle un día a un mes que ya está en su valor máximo. Estos excedentes requieren que el mes se incremente. Si el mes es ya el mes 12, el año también deberá ser incrementado.

El operador de preincremento homónimo devuelve una copia incrementada del objeto real. Esto ocurre porque el objeto real, `* this`, es regresado como un `Date`. Esto de hecho invoca al constructor de copia.

Resulta un poco más complejo hacer la homonimia del operador de postincremento. A fin de emular el efecto de postincremento, debemos regresar una copia del objeto `Date` sin incrementar. Al introducir el `operator++`, guardamos en `temp` el objeto actual (`* this`). A continuación llamamos a `helpIncrement` para incrementar el objeto. Por último regresamos la copia sin incrementar del objeto en `temp`.

```
// DATE1.H
// Definition of class Date
#ifndef DATE1_H
#define DATE1_H
#include <iostream.h>

class Date {
    friend ostream &operator<<(ostream &, const Date &);
public:
    Date(int m = 1, int d = 1, int y = 1900); // def. constructor
    void setDate(int, int, int); // set the date
    Date operator++(); // preincrement
    Date operator++(int); // postincrement
    Date &operator+=(int); // add days, modify object
private:
    int month;
    int day;
    int year;
    static int days[]; // array of days per month
    void helpIncrement(); // utility function
};

#endif
```

Fig. 18.6 Definición de la clase `Date` (parte 1 de 6).

```
// DATE1.CPP
// Member function definitions for Date class
#include <iostream.h>
#include "date1.h"

// Initialize static member at file scope;
// one class-wide copy.
int Date::days[] = {0, 31, 28, 31, 30, 31, 30,
                    31, 31, 30, 31, 30, 31};

// Date constructor
Date::Date(int m, int d, int y) { setDate(m, d, y); }

// Set the date
void Date::setDate(int mm, int dd, int yy)
{
    month = (mm >= 1 && mm <= 12) ? mm : 1;
    year = (yy >= 1900 && yy <= 2100) ? yy : 1900;

    // test for a leap year
    if (month == 2 && year % 4 == 0 && (year % 400 == 0 ||
                                        year % 100 != 0))
        day = (dd >= 1 && dd <= 29) ? dd : 1;
    else
        day = (dd >= 1 && dd <= days[month]) ? dd : 1;
}
```

Fig. 18.6 Definición de función miembro para la clase `Date` (parte 2 de 6).

```

// Preincrement operator overloaded as a member function.
Date Date::operator++()
{
    helpIncrement();
    return *this; // value return; not a reference return
}

// Postincrement operator overloaded as a member function.
// Note that the dummy integer parameter does not have a
// parameter name.
Date Date::operator++(int)
{
    Date temp = *this;
    helpIncrement();

    // return non-incremented, saved, temporary object
    return temp; // value return; not a reference return
}

// Add a specific number of days to a date
Date &Date::operator+=(int additionalDays)
{
    for (int i = 1; i <= additionalDays; i++)
        helpIncrement();

    return *this; // enables concatenation
}

// Function to help increment the date
void Date::helpIncrement()
{
    // test for a leap year first
    if (month == 2 && day < 29 && year % 4 == 0 &&
        (year % 400 == 0 || year % 100 != 0))
        ++day;
    else if (day < Date::days[month]) // not last day of month
        ++day;
    else if (month < 12) { // last day of month < December
        ++month;
        day = 1;
    }
    else { // December 31
        month = 1;
        day = 1;
        ++year;
    }
}

```

Fig. 18.6 Definición de función miembro para la clase Date (parte 3 de 6).

```

// Overloaded output operator
ostream &operator<<(ostream &output, const Date &d)
{
    static char *monthName[13] = {"", "January",
        "February", "March", "April", "May", "June",
        "July", "August", "September", "October",
        "November", "December"};

    output << monthName[d.month] << ' '
        << d.day << ", " << d.year;

    return output; // enables concatenation
}

```

Fig. 18.6 Definición de función miembro para la clase Date (parte 4 de 6).

```

// FIG18_6.CPP
// Driver for class Date
#include <iostream.h>
#include "date1.h"

main()
{
    Date d1, d2(12, 27, 1992), d3(0, 99, 8045);
    cout << "d1 is " << d1
        << "\nd2 is " << d2
        << "\nd3 is " << d3;

    cout << "\n\nd2 += 7 is " << (d2 += 7) << "\n\n";

    d3.setDate(2, 28, 1992);
    cout << " d3 is " << d3;
    cout << "\n++d3 is " << ++d3 << "\n\n";

    Date d4(3, 18, 1969);

    cout << "Testing the preincrement operator:\n"
        << " d4 is " << d4
        << "\n++d4 is " << ++d4
        << "\n d4 is " << d4;

    cout << "\n\nTesting the postincrement operator:\n"
        << " d4 is " << d4
        << "\nd4++ is " << d4++
        << "\n d4 is " << d4 << endl;

    return 0;
}

```

Fig. 18.6 Manejador para la clase Date (parte 5 de 6).

```

d1 is January 1, 1900
d2 is December 27, 1992
d3 is January 1, 1900

d2 += 7 is January 3, 1993

d3 is February 28, 1992
++d3 is February 29, 1992

Testing the preincrement operator:
d4 is March 18, 1969
++d4 is March 19, 1969
d4 is March 19, 1969

Testing the postincrement operator:
d4 is March 19, 1969
d4++ is March 19, 1969
d4 is March 20, 1969

```

Fig. 18.6 Salida correspondiente al manejador de la clase `Date` (parte 6 de 6).

### Resumen

- En C++ el operador `<<` es utilizado para varios fines, como operador de inserción de flujo y como operador de desplazamiento a la izquierda. Esto es un ejemplo de homonimia de operadores. De igual forma, `>>` también tiene homónimo; se utiliza tanto como operador de extracción de flujo, como operador de desplazamiento a la derecha.
- En forma más general, C++ le permite al programador hacer la homonimia de la mayor parte de los operadores, para que sean sensibles al contexto en el cual están siendo utilizados. El compilador genera el código apropiado basándose en la forma en la cual cada operador es utilizado.
- La homonimia de operadores contribuye a la extensibilidad de C++.
- Se lleva a cabo la homonimia de operadores al escribir una definición de función (incluyendo un encabezado y un cuerpo). El nombre de la función se convierte en la palabra reservada **operator**, seguida por el símbolo correspondiente al operador homónimo.
- Para utilizar un operador sobre objetos de clase, dicho operador *debe* ser un operador homónimo con dos excepciones. El operador de asignación (`=`) puede ser utilizado con cualquier clase para llevar a cabo una copia a nivel de miembro por omisión, sin necesidad de tener homónimo. El operador de dirección (`&`) también puede ser utilizado sobre objetos de cualquier clase sin tener homónimo; sólo devuelve la dirección en la memoria del objeto.
- Al hacer la homonimia de operadores la idea subyacente es disponer de las mismas expresiones concisas para tipos definidos por usuario que C++ proporciona con su rica colección de operadores para tipos incorporados.
- La mayor parte de los operadores de C++ pueden tener homónimos.

- La homonimia de un operador no puede modificar su precedencia y asociatividad.
- En un operador homónimo no se pueden utilizar argumentos por omisión.
- No es posible modificar el número de operandos que un operador toma: los operadores unarios homónimos se conservan como operadores unarios; los operadores binarios homónimos se conservan como operadores binarios.
- No es posible crear símbolos para operadores nuevos; sólo se puede hacer la homonimia de operadores existentes.
- El significado de cómo opera un operador sobre objetos de tipos incorporados no puede ser modificado mediante la homonimia.
- Al hacer la homonimia de `()`, `[]`, `->`, `o =`, la función operador homónima debe ser declarada como miembro de clase.
- Las funciones operador pueden ser funciones miembro o funciones no miembro.
- Cuando una función operador es puesta en práctica como función miembro, el operando más a la izquierda debe ser un objeto de clase (o una referencia a un objeto de clase) de la clase del operador.
- Si el operando a la izquierda tiene que ser un objeto de clase distinta, esta función operador debe ser puesta en práctica como una función no miembro.
- Las funciones miembro de operador son llamadas sólo cuando el operando izquierdo de un operador binario es en específico un objeto de dicha clase o cuando el único operando de un operador unario es un objeto de dicha clase.
- Se podría escoger una función no miembro para hacer la homonimia de un operador, a fin de permitir que el operador sea conmutativo.
- Se puede hacer la homonimia de un operador unario como función miembro no estática sin argumentos o como función no miembro con un argumento; dicho argumento debe ser un objeto de un tipo definido por usuario o una referencia a un objeto de un tipo definido por usuario.
- Se puede hacer la homonimia de un operador binario como función miembro no estática con un argumento, o como función no miembro con dos argumentos (uno de los cuales debe ser un objeto de clase o una referencia a un objeto de clase).
- El operador de subíndice de arreglo `[]` no está restringido en su uso sólo con arreglos; puede ser utilizado para seleccionar elementos de otros tipos de clases contenedor como son listas enlazadas, cadenas, diccionarios y demás. Además, ya no es necesario que los subíndices sean enteros, también pueden ser utilizados caracteres o cadenas, por ejemplo.
- Un constructor de copia se utiliza para inicializar un objeto con otro objeto de la misma clase. Los constructores de copia también son invocados siempre que se requiera de la copia de un objeto, como es el caso en la llamada por valor o al regresar un valor de una función llamada.
- El compilador no sabe en forma automática cómo convertir entre tipos definidos por usuario y tipos incorporados. El programador debe especificar en forma explícita cómo ocurrirán dichas conversiones. Estas conversiones pueden ser llevadas a cabo mediante constructores de conversión (es decir, constructores de un argumento) que sólo convierten objetos de otros tipos en objetos de una clase particular.
- Un operador de conversión (u operador cast) puede ser utilizado para convertir un objeto de una clase a un objeto de otra o a un objeto de un tipo incorporado. Dicho operador de conversión

debe ser una función miembro no estática; este tipo de operador de conversión no puede ser una función amigo.

- Un constructor de conversión es un constructor de un argumento, que es utilizado para convertir el argumento en el objeto de la clase del constructor. El compilador puede llamar de forma implícita a un constructor de este tipo.
- El operador de asignación es el operador homónimo más frecuente. Por lo regular se utiliza para asignar un objeto a otro objeto de la misma clase, pero mediante el uso de los constructores de conversión; también puede ser utilizado para asignaciones entre clases.
- Si no se define un operador de asignación homónimo, la asignación se permitirá, pero será por omisión a una copia a nivel de miembro de cada miembro de datos. En algunos casos esto es aceptable. En el caso de objetos que contengan apuntadores a almacenamiento dinámico asignado, la copia a nivel de miembro dará como resultado dos objetos distintos apuntando al mismo almacenamiento asignado dinámicamente. Cuando se llame el destructor correspondiente a cualquiera de estos objetos, se liberará el almacenamiento dinámico asignado. Si el otro objeto entonces quiere referirse a dicho almacenamiento, el resultado quedará indefinido.
- Para hacer la homonimia del operador de incremento y permitir el uso tanto de preincremento como de postincremento, cada función de operador homónima debe tener una signatura distinta o diferente, de tal forma, que el compilador pueda ser capaz de determinar qué versión de ++ debe utilizar. Las versiones prefijas se hacen homónimas de igual forma que cualquier operador unario prefijo. El proporcionar una firma única a la función de operador de postincremento se consigue incluyendo un segundo argumento que debe ser del tipo `int`. De hecho, el usuario no proporciona un valor para este argumento entero especial. Aparece ahí simplemente para ayudar al compilador a distinguir entre versiones prefijas y postfijas de los operadores de incremento y de decremento.

## Terminología

tipo incorporado  
clase **Array**  
clase **Date**  
clase **String**  
llamadas de función concatenadas  
resolución de conversión de ambigüedad  
constructor de conversión  
función de conversión  
operador de conversión  
conversiones entre clases de tipos diferentes  
conversiones entre tipos y clases incorporados  
constructor de copia  
conversiones explícitas de tipo (mediante `cast`)  
operador homónimo de función amigo  
homonimia de función  
conversiones implícitas de tipo  
operador homónimo de función miembro  
operadores no sujetos de homonimia  
palabra reservada **operator**

homonimia de operadores  
operadores como funciones  
operadores capaces de homonimia  
operador homónimo `==`  
operador homónimo `!=`  
operador homónimo `<`  
operador homónimo `<=`  
operador homónimo `>`  
operador homónimo `>=`  
operador de asignación homónimo `(=)`  
operador homónimo `[]`  
hacer la homonimia  
homonimia de un operador binario  
homonimia de un operador unario  
homonimia de un operador postfijo  
homonimia de un operador prefijo  
conversión definida por usuario  
tipo definido por usuario

## Errores comunes de programación

- 18.1 Intentar crear operadores nuevos.
- 18.2 Intentar modificar cómo funciona un operador con objetos de tipos incorporados.
- 18.3 Suponer que hacer la homonimia de un operador (como `+`) hace de forma automática la homonimia de sus operadores relacionados (como `+=`). Los operadores sólo pueden tener homónimos en forma explícita; la homonimia implícita no existe.
- 18.4 No probar la existencia y evitar la autoasignación cuando se hace la homonimia del operador de asignación de una clase que incluya un apuntador a almacenamiento dinámico.
- 18.5 No proporcionar un operador de asignación homónimo y un constructor de copia para una clase, cuando los objetos de dicha clase contengan apuntadores a almacenamiento dinámico asignado.

## Prácticas sanas de programación

- 18.1 Utilice la homonimia de operadores cuando ésta haga más claro un programa si efectúa las mismas operaciones mediante llamadas de función explícitas.
- 18.2 Evite un uso excesivo o inconsistente de la homonimia de operadores, ya que ello puede hacer que un programa resulte críptico o difícil de leer.
- 18.3 Haga la homonimia de operadores para llevar a cabo la misma función o funciones bastante similares sobre objetos de clase que dichos operadores ejecutan sobre objetos de tipos incorporados.
- 18.4 Antes de escribir programas en C++ utilizando operadores homónimos, consulte los manuales de C++ correspondientes a su compilador, para informarse de las varias restricciones y requisitos únicos a operadores particulares.
- 18.5 Al hacer la homonimia de operadores unarios, es preferible hacer la funciones operador miembros de clase en vez de funciones amigo no miembro. Esta es una solución más limpia. Las funciones amigo y las clases amigo deberán evitarse, salvo que sean en lo absoluto necesarias. La utilización de amigos viola el encapsulado de una clase.
- 18.6 Un destructor, el operador de asignación, y un constructor de copia para una clase por lo general se proporcionan en grupo.

## Sugerencias de rendimiento

- 18.1 Es posible hacer la homonimia de un operador como función no miembro, y no amigo, pero dicha función con la necesidad de tener acceso a datos privados o protegidos de una clase requeriría utilizar las funciones `set` o `get` proporcionadas en la interfaz pública de la clase. La sobrecarga de llamar a estas funciones causaría bajo rendimiento.
- 18.2 Usar el operador de concatenación homónimo `+=` que toma un solo argumento del tipo `const char *` da resultados más eficaces que primero tener que efectuar la conversión implícita y a continuación la concatenación. Las conversiones implícitas requieren de menos código y causan menos errores.

## Observaciones de ingeniería de software

- 18.1 En una función operador, por lo menos un argumento debe ser un objeto de clase o una referencia a un objeto de clase. Esto impedirá que los programadores modifiquen cómo funcionan los operadores sobre objetos de tipos incorporados.
- 18.2 Se pueden añadir nuevas capacidades de entrada/salida a C++ correspondientes a tipos definidos por usuario, sin modificar las declaraciones o los miembros de datos privados correspondientes a la clase `ostream` o a la clase `istream`. Esto promueve la extensibilidad del lenguaje de programación C++ —uno de los aspectos más atractivos de C++.

- 18.3 Es posible evitar que un objeto de clase sea asignado a otro. Esto se lleva a cabo definiendo el operador de asignación como miembro privado de la clase.

### Ejercicios de autoevaluación

- 18.1 Llene cada uno de los siguientes espacios en blanco:
- Suponga que **a** y **b** son variables enteras y que formamos la suma **a + b**. Ahora suponga que **c** y **d** son variables de punto flotante y que formamos la suma **c + d**. Los dos operadores **+** están siendo utilizados aquí con claridad para fines distintos. Esto es un ejemplo de \_\_\_\_\_.
  - La palabra reservada \_\_\_\_\_ introduce una definición de función de operador homónimo.
  - Para utilizar operadores sobre objetos de clase, deben tener homónimos a excepción de los operadores \_\_\_\_\_ y \_\_\_\_\_.
  - La \_\_\_\_\_ y \_\_\_\_\_ de un operador no pueden ser modificadas mediante la homonimia.
- 18.2 Explique los varios significados de los operadores **<< y >>** en C++.
- 18.3 ¿En que contexto pudiera ser utilizado el nombre **operator/** en C++?
- 18.4 (Verdadero/falso). En C++ solo los operadores existentes pueden tener homónimos.
- 18.5 ¿Cómo se compara la precedencia de un operador homónimo en C++ con la precedencia del operador original?

### Respuestas a los ejercicios de autoevaluación

- 18.1 a) homonimia de operador. b) **operator**. c) asignación (**=**), dirección (**&**). d) precedencia, asociatividad.
- 18.2 El operador **>>** es, dependiendo del contexto, a la vez operador de desplazamiento a la derecha y operador de extracción de flujo. El operador **<<** es a la vez operador de desplazamiento a la izquierda y operador de inserción de flujo, dependiendo del contexto.
- 18.3 Para la homonimia de operadores: sería el nombre de una función que proporcionaría una nueva versión del operador **/**.
- 18.4 Verdadero.
- 18.5 Idéntico.

### Ejercicios

- 18.6 De tantos ejemplos como pueda de la homonimia de operadores implícita en C. De tantos ejemplos como pueda de la homonimia de operadores implícita en C++. De un ejemplo razonable de una situación en la cual pudiera usted desear hacer la homonimia explícita de un operador en C++.

- 18.7 Los operadores C++ que no pueden tener homónimos son \_\_\_\_\_, \_\_\_\_\_, \_\_\_\_\_, \_\_\_\_\_ y \_\_\_\_\_.

- 18.8 La concatenación de cadenas requiere de dos operandos —las dos cadenas que han de concatenarse. En el texto mostramos cómo poner en práctica un operador de concatenación homónimo, que concatena el segundo objeto **string** a la derecha del primer objeto **string**, modificando así el primer objeto **string**. En algunas aplicaciones, es deseable producir un objeto concatenado **string** sin modificar los dos argumentos **string**. Diseñe un **operator+** para permitir operaciones como

```
string1 = string2 + string3;
```

- 18.9 (Ejercicio de homonimia de operadores final.) Para poder valorizar el cuidado que debe ponerse en la selección de operadores para su homonimia, liste cada uno de los operadores que se puedan hacer

homónimos en C++ y por cada uno de ellos liste un significado posible (o varios, si es apropiado) para cada una de las distintas clases que ha estudiado en este curso. Sugerimos que pruebe:

- Arreglos
- Pilas
- Cadenas

Una vez hecho lo anterior, comente sobre qué operadores parecen tener significado para una amplia variedad de clases. ¿Qué operadores parecen tener poco valor para homonimia? ¿Qué operadores parecen ambiguos?

- 18.10 Ahora ejecute el proceso descrito en el programa anterior pero a la inversa. Liste cada uno de los operadores capaces de homonimia en C++. Para cada uno de ellos, liste lo que usted sienta es quizás la "operación óptima" para la cual dicho operador debería ser utilizado. Si existen varias operaciones excelentes, listelas todas.

- 18.11 (Proyecto). C++ es un lenguaje en evolución, y continuamente se están desarrollando nuevos lenguajes. ¿Qué operadores adicionales recomendaría usted añadir a C++ o a un lenguaje similar a C++, que apoyasen tanto a la programación procedural como a la programación orientada a objetos? Escriba una cuidadosa justificación. Pudiera inclusive considerar enviar sus sugerencias al Comité ANSI C++.

- 18.12 Un buen ejemplo de la homonimia del operador de llamada de función (**()**) es permitir la forma más común de dobles subíndices de arreglo. En vez de decir

```
chessBoard[row][column]
```

en relación con un arreglo de objetos, haga la homonimia del operador de llamada de función para que pueda utilizarse la forma alterna

```
chessBoard(row, column)
```

- 18.13 Haga la homonimia del operador de subíndice para regresar un miembro dado en una lista enlazada.

- 18.14 Haga la homonimia del operador de subíndice para regresar el elemento más grande de una colección, el segundo más grande, el tercero más grande, etcétera.

- 18.15 Considere la clase **Complex** mostrada en la figura 18.7. La clase permite operaciones de los números llamados *complejos*. Estos son números de forma **realPart + imaginaryPart \* i**, donde **i** tiene el valor:

$$\sqrt{-1}$$

- Modifique la clase, para permitir la entrada y salida de números complejos, mediante los operadores homónimos **>>** y **<<**, respectivamente (deberá eliminar la función de impresión de la clase).
- Haga la homonimia del operador de multiplicación, para permitir la multiplicación de números complejos, como en álgebra.
- Haga la homonimia de los operadores **==** y **!=**, para permitir comparaciones de números complejos.

- 18.16 Una máquina con enteros de 32 bits puede representar enteros en el rango de aproximadamente -2 mil millones hasta +2 mil millones. Esta restricción de tamaño fijo rara vez causa problemas. Pero existen muchas aplicaciones en las cuales nos gustaría ser capaces de utilizar un rango mucho más amplio de enteros. Para esto fue construido C++, es decir para crear nuevos tipos poderosos de datos. Considere la clase **HugeInt** de la figura 18.8. Estudie cuidadosamente esta clase, y después

- Describa con precisión cómo funciona.
- ¿Qué restricciones tiene la clase?
- Modifique la clase para que pueda procesar enteros arbitrariamente grandes. (Sugerencia: utilice una lista enlazada para representar a **HugeInt**.)
- Haga la homonimia del operador de multiplicación **\***.
- Haga la homonimia del operador de división **/**.
- Haga la homonimia de todos los operadores relacionales y de igualdad.

```
// COMPLEX1.H
// Definition of class Complex
#ifndef COMPLEX1_H
#define COMPLEX1_H

class Complex {
public:
    Complex(double = 0.0, double = 0.0);    // constructor
    Complex operator+(const Complex &) const; // addition
    Complex operator-(const Complex &) const; // subtraction
    Complex &operator=(const Complex &);    // assignment
    void print() const;                     // output
private:
    double real;        // real part
    double imaginary;   // imaginary part
};

#endif
```

Fig. 18.7 Definición de la clase Complex (parte 3 de 5).

```
// COMPLEX1.CPP
// Member function definitions for class Complex
#include <iostream.h>
#include "complex1.h"

// Constructor
Complex::Complex(double r, double i)
{
    real = r;
    imaginary = i;
}

// Overloaded addition operator
Complex Complex::operator+(const Complex &operand2) const
{
    Complex sum;
    sum.real = real + operand2.real;
    sum.imaginary = imaginary + operand2.imaginary;
    return sum;
}

// Overloaded subtraction operator
Complex Complex::operator-(const Complex &operand2) const
{
    Complex diff;
    diff.real = real - operand2.real;
    diff.imaginary = imaginary - operand2.imaginary;
    return diff;
}
```

Fig. 18.7 Definiciones de función miembro para la clase Complex (parte 2 de 5).

```
// Overloaded = operator
Complex& Complex::operator=(const Complex &right)
{
    real = right.real;
    imaginary = right.imaginary;
    return *this;    // enables concatenation
}

// Display a Complex object in the form: (a, b)
void Complex::print() const
{ cout << '(' << real << ", " << imaginary << ')'; }
```

Fig. 18.7 Definiciones de función miembro para la clase Complex (parte 3 de 5).

```
// FIG18_7.CPP
// Driver for class Complex
#include <iostream.h>
#include "complex1.h"

main()
{
    Complex x, y(4.3, 8.2), z(3.3, 1.1);

    cout << "x: ";
    x.print();
    cout << "\ny: ";
    y.print();
    cout << "\nz: ";
    z.print();

    x = y + z;
    cout << "\n\nx = y + z:\n";
    x.print();
    cout << " = ";
    y.print();
    cout << " + ";
    z.print();

    x = y - z;
    cout << "\n\nx = y - z:\n";
    x.print();
    cout << " = ";
    y.print();
    cout << " - ";
    z.print();
    cout << '\n';

    return 0;
}
```

Fig. 18.7 Manejador para la clase Complex (parte 4 de 5).

```

x: (0, 0)
y: (4.3, 8.2)
z: (3.3, 1.1)

x = y + z:
(7.6, 9.3) = (4.3, 8.2) + (3.3, 1.1)

x = y - z:
(1, 7.1) = (4.3, 8.2) - (3.3, 1.1)

```

Fig. 18.7 Salida del manejador para la clase **Complex** (parte 5 de 5).

```

// HUGEINT1.H
// Definition of the HugeInt class
#ifndef HUGEINT1_H
#define HUGEINT1_H

#include <iostream.h>

class HugeInt {
    friend ostream &operator<<(ostream &, HugeInt &);
public:
    HugeInt(long = 0);           // conversion constructor
    HugeInt(const char *);       // conversion constructor
    HugeInt operator+(HugeInt &); // addition
private:
    short integer[30];
};

#endif

```

Fig. 18.8 Clase de grandes enteros definida por usuario (parte 1 de 4).

```

// HUGEINT1.CPP
// Member and friend function definitions for class HugeInt
#include <string.h>
#include "hugeint1.h"

// Conversion constructor
HugeInt::HugeInt(long val)
{
    for (int i = 0; i <= 29; i++)
        integer[i] = 0; // initialize array to zero

    for (i = 29; val != 0 && i >= 0; i--) {
        integer[i] = val % 10;
        val /= 10;
    }
}

```

Fig. 18.8 Clase de grandes enteros definida por usuario (parte 2 de 4).

```

HugeInt::HugeInt(const char *string)
{
    int i, j;

    for (i = 0; i <= 29; i++)
        integer[i] = 0;

    for (i = 30 - strlen(string), j = 0; i <= 29; i++, j++)
        integer[i] = string[j] - '0';
}

// Addition
HugeInt HugeInt::operator+(HugeInt &op2)
{
    HugeInt temp;
    int carry = 0;

    for (int i = 29; i >= 0; i--) {
        temp.integer[i] = integer[i] + op2.integer[i] + carry;

        if (temp.integer[i] > 9) {
            temp.integer[i] %= 10;
            carry = 1;
        }
        else
            carry = 0;
    }

    return temp;
}

ostream& operator<<(ostream &output, HugeInt &num)
{
    for (int i = 0; (num.integer[i] == 0) && (i <= 29); i++)
        ; // skip leading zeros

    if (i == 30)
        output << 0;
    else
        for (; i <= 29; i++)
            output << num.integer[i];

    return output;
}

```

Fig. 18.8 Clase de grandes enteros definida por usuario (parte 3 de 4).

- 18.17 Cree una clase **rationalNumber** (fracciones) con las siguientes capacidades:
- Cree un constructor que impida la existencia de un denominador 0 en una fracción, que simplifique las fracciones que no estén en forma simplificada, y que evite denominadores negativos.
  - Haga la homonimia de los operadores de suma, resta, multiplicación y división para esta clase.
  - Haga la homonimia de los operadores relacionales y de igualdad para esta clase.



# 19

---

## Herencia

---

### Objetivos

- Ser capaz de crear nuevas clases heredando de clases existentes.
- Comprender como la herencia fomenta la reutilización del software.
- Comprender los conceptos de clases base y clases derivadas.
- Ser capaz de utilizar herencia múltiple para derivar una clase a partir de varias clases base.

*No diga que conoce por completo a otro, hasta que con él haya dividido una herencia.*

Johann Kasper Lavater

*Este método es para definir como el número de una clase la clase de todas las clases similares a la clase dada.*

Bertrand Russell

*Un mazo de cartas fue diseñado como la más pura de las jerarquías, siendo cada una de las cartas superior a todas las que están debajo de ella, y lacayo de las que están por encima.*

Ely Culbertson

*Aunque es muy bueno heredar una biblioteca, mejor aún es reunirla.*

Augustine Birrell

*Ahorre autoridad base de libros de terceros.*

William Shakespeare

*Love's Labour's Lost*

## Sinopsis

- 19.1 Introducción
- 19.2 Clases base y clases derivadas
- 19.3 Miembros protegidos
- 19.4 Cómo hacer la conversión explícita (cast) de apuntadores de clase base a apuntadores de clase derivada
- 19.5 Cómo utilizar funciones miembro
- 19.6 Cómo redefinir los miembros de clase base en una clase derivada
- 19.7 Clases base públicas, protegidas y privadas
- 19.8 Clases base directas y clases base indirectas
- 19.9 Cómo utilizar constructores y destructores en clases derivadas
- 19.10 Conversión implícita de objeto de clase derivada a objeto de clase base
- 19.11 Ingeniería de software con herencia
- 19.12 Composición en comparación con herencia
- 19.13 Relaciones "utiliza un" y "conoce un"
- 19.14 Estudio de caso: Point, Circle, Cylinder
- 19.15 Herencia múltiple

*Resumen • Terminología • Errores comunes de programación • Prácticas sanas de programación • Sugerencia de rendimiento • Observaciones de ingeniería de software • Ejercicios de autoevaluación • Respuestas a los ejercicios de autoevaluación • Ejercicios.*

### 19.1 Introducción

En éste y en el capítulo siguiente analizamos dos de las capacidades de mayor importancia que proporciona la programación orientada a objetos *herencia* y *polimorfismo*. La herencia es una forma de *reutilización del software*, en la cual se crean clases nuevas a partir de clases existentes, mediante la absorción de sus atributos y comportamientos, y embelleciendo éstos con las capacidades que las clases nuevas requieren. La reutilización del software ahorra tiempo en el desarrollo de programas, fomenta la reutilización de software de alta calidad, probado y depurado, y reduce problemas en un sistema después de convertido en funcional. Estas son excitantes posibilidades. El polimorfismo nos permite escribir programas en forma general, a fin de procesar una amplia variedad de clases relacionadas tanto existentes como aún sin especificar. La herencia y el polimorfismo son técnicas efectivas para enfrentarse con la complejidad del software.

Al crear una clase nueva, en vez de escribir en su totalidad miembros de datos y funciones miembro nuevos, el programador puede determinar que la clase nueva debe *heredar* los miembros de datos y las funciones miembro provenientes de una *clase base* ya definida. La clase nueva se

conoce como *clase derivada*. Cada clase derivada misma se convierte en candidato a clase base, para alguna clase derivada futura. Mediante la *herencia única*, una clase es derivada de una única clase base. Con la *herencia múltiple* una clase derivada hereda de múltiples clases base (posiblemente no relacionadas). A menudo una clase derivada añade miembros de datos y funciones miembro propias, por lo que en general una clase derivada es más grande que su clase base. Una clase derivada es más específica que su clase base y representa un grupo más pequeño de objetos. Con la herencia única, la clase derivada se inicia en esencia de la misma forma que la clase base. La fuerza real de la herencia proviene de la capacidad de definir en la clase derivada, adiciones, reemplazos o refinamientos de las características heredadas de la clase base.

Cada objeto de una clase derivada también es un objeto de la clase base de dicha clase derivada. Sin embargo; la inversa no es verdad, los objetos de la clase base no son objetos de la clase derivada de dicha clase base. Nos aprovecharemos de esta relación "un objeto de clase derivada es un objeto de clase base" para ejecutar algunas manipulaciones interesantes. Por ejemplo, podemos unir una gran variedad de objetos distintos relacionados mediante la herencia en una lista enlazada de objetos de clase base. Esto permite que una variedad de objetos se procesen de forma general. Como veremos en éste y en el siguiente capítulo, esto es un impulso clave a la programación orientada a objetos.

En este capítulo añadimos una nueva forma de control de acceso de miembros, es decir el acceso protegido. Para el acceso a los miembros de clase base protegidos, las clases derivadas y sus amigos reciben un trato favorecido en relación con otras funciones.

La experiencia en la elaboración de sistemas de software indica que grandes porciones de código se refieren a casos especiales muy relacionados. En dichos sistemas se hace difícil visualizar la "imagen general" porque el diseñador y el programador se han visto preocupados por los casos especiales. La programación orientada a objetos proporciona varias formas de "ver el bosque a través de los árboles" —un proceso conocido a veces como *abstracción*.

Si un programa está lleno de casos especiales muy relacionados, entonces es común ver enunciados **switch**, que distinguen entre casos especiales, que proporcionan la lógica de proceso necesaria para manejar cada caso en forma individual. En el capítulo 20, mostraremos cómo utilizar la herencia y el polimorfismo para reemplazar la lógica **switch** por una lógica más simple.

Distinguimos entre *relaciones "es un"* y *relaciones "tiene un"*. "Es un" es herencia. En una relación "es un", un objeto de un tipo de clase derivada también puede ser tratado como un objeto del tipo de clase base. "Tiene un" es composición (véase la figura 17.4). En una relación "tiene un" un objeto de clase tiene como miembros uno o más objetos de otras clases.

Una clase derivada no puede tener acceso a los miembros privados de su clase base; permitirlo violaría el encapsulado de la clase base. Una clase derivada puede, sin embargo, tener acceso a los miembros públicos y protegidos de su clase base. Los miembros de clase base que no deban ser accesibles a una clase derivada mediante la herencia, en la clase base se declararán privados. Una clase derivada puede tener acceso a los miembros privados de la clase base mediante funciones de acceso, provistas en la interfaz pública de la clase base. Un problema, tratándose de la herencia, es que una clase derivada puede heredar funciones miembro públicas que no requiera, y que expresamente no tendría. Cuando un miembro de clase base no es apropiado para una clase derivada, dicho miembro puede ser redefinido en la clase derivada con una puesta en práctica apropiada.

De lo más excitante resulta la idea que las clases nuevas pueden heredar de *bibliotecas de clase* existentes. Las organizaciones desarrollan sus propias bibliotecas de clase y pueden aprovechar de otras bibliotecas disponibles en todo el mundo. La perspectiva es que, en algún momento, el software será elaborado a partir de *componentes reutilizables estandarizados*, de la

misma forma que, hoy día, a menudo es construido el hardware. Esto ayudará a estar a la altura del reto de desarrollar el software más y más poderoso que en el futuro necesitaremos.

19.2 Clases base y clases derivadas

A menudo un objeto de clase en realidad también “es un” objeto de otra clase. Un rectángulo *es* un cuadrilátero (como lo es un cuadrado, un paralelogramo y un trapezoide). Entonces, la clase **Rectangle** puede decirse que *hereda* de la clase **Quadrilateral**. En este contexto, la clase **Quadrilateral** se conoce como *clase base* y la clase **Rectangle** se conoce como *clase derivada*. Un rectángulo *es* un tipo específico de un cuadrilátero, pero sería incorrecto decir que un cuadrilátero *es* un rectángulo. En la figura 19.1 se muestran varios ejemplos sencillos de herencia.

Otros lenguajes de programación orientados a objetos, como Smalltalk, utilizan diferentes terminologías: en la herencia, la clase base se conoce como *superclase* y la clase derivada como *subclase*. Dado que por lo general la herencia produce clases derivadas de tamaño mayor que las clases base, los términos superclase y subclase parecen inapropiados; evitaremos utilizar estos términos.

La herencia forma estructuras jerárquicas de apariencia arborescente. Una clase base existe en una relación jerárquica con sus clases derivadas. Ciertamente una clase puede existir por sí misma, pero es cuando una clase es utilizada mediante el mecanismo de la herencia, que ésta se convierte en clase base, proveyendo atributos y comportamientos a otras clases, o la clase se convierte en una clase derivada, que hereda dichos atributos y comportamientos.

Desarrollemos una jerarquía simple de herencia. Una comunidad universitaria típica está formada por miles de personas que son miembros de la misma. Estas personas son los empleados y los estudiantes. Los empleados son o miembros de la facultad o personal de oficinas. Los miembros de la facultad son o administradores (rectores o coordinadores de departamentos) o la facultad de profesores. Esto da como resultado la jerarquía de herencia que se muestra en la figura 19.2.

| Clase base | Clases derivadas    |
|------------|---------------------|
| Student    | CommuterStudent     |
|            | ResidentStudent     |
| Shape      | Circle              |
|            | Triangle            |
|            | Rectangle           |
| Loan       | CarLoan             |
|            | HomeImprovementLoan |
|            | MortgageLoan        |
| Employee   | FacultyMember       |
|            | StaffMember         |
| Account    | CheckingAccount     |
|            | SavingsAccount      |

Fig. 19.1 Algunos ejemplos simples de herencia.

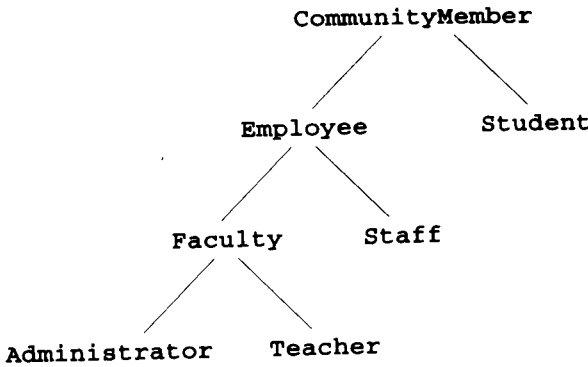


Fig. 19.2 Una jerarquía de herencia para los miembros de una comunidad universitaria.

Otra jerarquía de herencia sustancial es la jerarquía **Shape** de la figura 19.3. Un comentario común entre aquellos alumnos que estudian por primera vez la programación orientada a objetos, es que en el mundo real existen ejemplos abundantes de jerarquías. Es simple que a estos alumnos jamás se les ha pedido que categoricen el mundo real de esta forma, por lo que deben hacer algunos ajustes en su forma de pensar.

Para especificar que la clase **CommissionWorker** se deriva de la clase **Employee**, típicamente la clase **CommissionWorker** sería definida como sigue:

```
class CommissionWorker : public Employee {  
    ...  
};
```

Esto se conoce como *herencia pública* y es el tipo que más probablemente utilice el lector. También analizaremos *herencia privada* y *herencia protegida*. En el caso de la herencia pública, los miembros públicos y protegidos de la clase base son heredados como miembros públicos y protegidos de la clase derivada, respectivamente.

Es posible tratar en forma similar a los objetos de la clase base y a los objetos de la clase derivada. El estado común es expresado en los atributos y comportamientos de la clase base. Los objetos de cualquier clase derivada pública originados de una clase base común, pueden todos ellos ser tratados como objetos de dicha clase base.

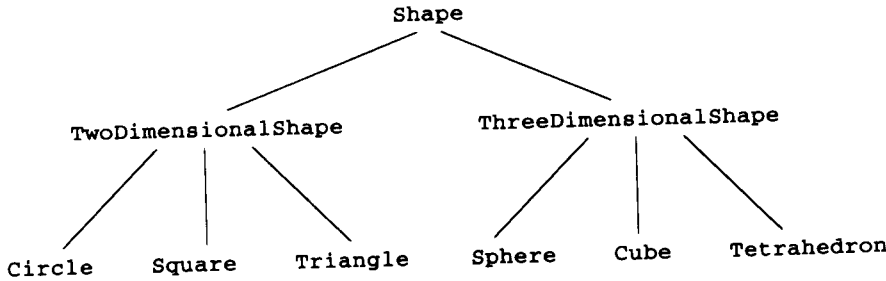


Fig. 19.3 Una porción de la jerarquía de clase Shape.

Veremos muchos ejemplos en los cuales aprovecharemos esta relación con una facilidad de programación no disponible en lenguajes no orientados a los objetos como es C.

### 19.3 Miembros protegidos

Los miembros públicos de una clase base son accesibles a todas las funciones en el programa. Los miembros privados de clase base son accesibles sólo para las funciones miembro y los amigos de la clase base.

El acceso protegido sirve como nivel intermedio de protección entre el acceso público y el acceso privado. Los miembros protegidos de clase base son accesibles sólo por miembros y amigos de la clase base, y por miembros y amigos de las clases derivadas.

Los miembros de clases derivadas pueden hacer referencia a miembros públicos y protegidos de la clase base sólo utilizando los nombres de los miembros. No es necesario utilizar el operador de resolución de alcance. Se supone el objeto actual.

### 19.4 Cómo hacer la conversión explícita (cast) de apuntadores de clase base a apuntadores de clase derivada

Un objeto de clase derivada pública también puede ser tratado como objeto de su clase base correspondiente. Esto permite algunas manipulaciones interesantes. Por ejemplo, a pesar del hecho que objetos de una variedad de clases derivadas de clase base particular pudieran ser muy distintos unos de los otros, aún así, podemos crear una lista enlazada de los mismos otra vez, siempre y cuando los tratemos como objetos de clase base. Pero lo inverso no es cierto: un objeto de clase base no es también automáticamente un objeto de clase derivada.

#### Error común de programación 19.1

*Puede causar errores tratar un objeto de clase base como si fuera un objeto de clase derivada.*

El programa podrá, sin embargo, utilizar un especificador de conversión explícita (cast) para convertir uno de clase base a un apuntador de clase derivada. Pero tenga cuidado si dicho apuntador debe ser desreferenciado, entonces primero deberá hacer que señale a un objeto del tipo de clase derivada.

#### Error común de programación 19.2

*Efectuar la conversión explícita de un apuntador de clase base que señala a un objeto de clase base a un apuntador de clase derivada y a continuación, hacer referencia a miembros de clase derivada que no existen en dicho objeto.*

Nuestro primer ejemplo aparece en la figura 19.4. partes 1 hasta la 5. Las partes 1 y 2 muestran la definición de clase **Point** y las definiciones de funciones miembro **Point**. Las partes 3 y 4 muestran la definición de clase **Circle** y la definición de función miembro **Circle**. La parte 5 muestra un programa manejador, en el cual demostramos la asignación de apuntadores de clase derivada a apuntadores de clase base, y la conversión explícita (cast) de apuntadores de base clase a apuntadores de clase derivada.

Examinemos primero la definición de clase **Point** (figura 19.4, parte 1). La interfaz pública a **Point** tiene las funciones miembro **setPoint**, **getX**, y **getY**. Los miembros de datos **x** y **y** de **Point** se definen como **protected**. Esto impide que los usuarios de los objetos **Point** tengan acceso directo a los datos, pero permite que las clases que se deriven de **Point** tengan acceso directo a los miembros de datos heredados. Si los datos fueran definidos como **private**, para tener acceso a los datos tendrían que ser utilizadas las funciones miembro públicas de **Point**.

Note que la función de operador de inserción de flujo homónima **Point** (figura 19.4 parte 2), es capaz de hacer referencia directa a las variables **x** y **y** porque éstas son miembros protegidos de la clase **Point**. Note también que es necesario hacer referencia a **x** y a **y** mediante objetos como en **p.x** y **p.y**. Esto es debido a que la función de operador de inserción de flujo homónima no es una función miembro de la clase **Circle**, sino que es un amigo de la clase.

```
// POINT.H
// Definition of class Point
#ifndef POINT_H
#define POINT_H

class Point {
    friend ostream &operator<<(ostream &, const Point &);
public:
    Point(float = 0, float = 0);    // default constructor
    void setPoint(float, float);    // set coordinates
    float getX() const { return x; } // get x coordinate
    float getY() const { return y; } // get y coordinate
protected:    // accessible by derived classes
    float x, y;    // x and y coordinates of the Point
};

#endif
```

Fig. 19.4 Definición de la clase **Point** (parte 1 de 5).

```
// POINT.CPP
// Member functions for class Point
#include <iostream.h>
#include "point.h"

// Constructor for class Point
Point::Point(float a, float b)
{
    x = a;
    y = b;
}

// Set x and y coordinates of Point
void Point::setPoint(float a, float b)
{
    x = a;
    y = b;
}

// Output Point (with overloaded stream insertion operator)
ostream &operator<<(ostream &output, const Point &p)
{
    output << '[' << p.x << ", " << p.y << ']' ;

    return output;    // enables concatenated calls
}
```

Fig. 19.4 Definiciones de función miembro para la clase **Point** (parte 2 de 5).

```
// CIRCLE.H
// Definition of class Circle
#ifndef CIRCLE_H
#define CIRCLE_H

#include <iostream.h>
#include <iomanip.h>
#include "point.h"

class Circle : public Point { // Circle inherits from Point
    friend ostream &operator<<(ostream &, const Circle &);
public:
    // default constructor
    Circle(float r = 0.0, float x = 0, float y = 0);

    void setRadius(float); // set radius
    float getRadius() const; // return radius
    float area() const; // calculate area
protected:
    float radius;
};

#endif
```

Fig. 19.4 Definición de la clase Circle (parte 3 de 5).

```
// CIRCLE.CPP
// Member function definitions for class Circle
#include "circle.h"

// Constructor for Circle calls constructor for Point
// with a member initializer then initializes radius.
Circle::Circle(float r, float a, float b)
    : Point(a, b) // call base-class constructor
{ radius = r; }

// Set radius of Circle
void Circle::setRadius(float r) { radius = r; }

// Get radius of Circle
float Circle::getRadius() const { return radius; }

// Calculate area of Circle
float Circle::area() const
{ return 3.14159 * radius * radius; }

// Output a Circle in the form:
// Center = [x, y]; Radius = #.##
ostream &operator<<(ostream &output, const Circle &c)
{
    output << "Center = [" << c.x << ", " << c.y
        << "]; Radius = " << setiosflags(ios::showpoint)
        << setprecision(2) << c.radius;

    return output; // enables concatenated calls
}
```

Fig. 19.4 Definición de función miembro para la clase Circle (parte 4 de 5).

```
// FIG19_4.CPP
// Casting base-class pointers to derived-class pointers

#include <iostream.h>
#include <iomanip.h>
#include "point.h"
#include "circle.h"

main()
{
    Point *pointPtr, p(3.5, 5.3);
    Circle *circlePtr, c(2.7, 1.2, 8.9);

    cout << "Point p: " << p << "\nCircle c: " << c << endl;

    // Treat a Circle as a Circle (with some casting)
    pointPtr = &c; // assign address of Circle to pointPtr
    circlePtr = (Circle *) pointPtr; // cast base to derived
    cout << "\nArea of c (via circlePtr): "
        << circlePtr->area() << endl;

    // DANGEROUS: Treat a Point as a Circle
    pointPtr = &p; // assign address of Point to pointPtr
    circlePtr = (Circle *) pointPtr; // cast base to derived
    cout << "\nRadius of object circlePtr points to: "
        << circlePtr->getRadius() << endl;

    return 0;
}
```

```
Point p: [3.5, 5.3]
Circle c: Center = [1.2, 8.9]; Radius = 2.70
Area of c (via circlePtr): 22.90
Radius of object circlePtr points to: 4.02e-38
```

Fig. 19.4 Conversión explícita de apunadores de clase base a apunadores de clase derivada (parte 5 de 5).

La clase **Circle** (figura 19.4, parte 3) hereda de la clase **Point** mediante herencia pública. Esto se especifica en la primera línea de la definición de clase

```
class Circle : public Point { // Circle inherits from Point
```

Los dos puntos (:) en el encabezado de la definición de clase indican herencia. La palabra reservada **public** indica el tipo de herencia (vea la sección 19.7). Todos los miembros de la clase **Point** son heredados a la clase **Circle**. Esto significa que la interfaz pública a **Circle** incluye la funciones miembro públicas de **Point**, así como las funciones miembros **Circle** de nombre **area**, **setRadius**, y **getRadius**.

El constructor **Circle** (figura 19.4, parte 4), deberá invocar al constructor **Point** para inicializar la porción de clase base de un objeto **Circle**. Esto se lleva a cabo con la sintaxis de inicializador de miembros que se presentó en el capítulo 17, como sigue

```
Circle::Circle(float r, float a, float b)
: Point(a, b) // call base-class constructor
```

La segunda línea del encabezado de función constructor invoca al constructor `Point` por su nombre. Los valores `a` y `b` son pasados del constructor `Circle` al constructor `Point`, a fin de inicializar los miembros de clase base `x` y `y`. Note que la función de operador de inserción de flujo homónima es capaz de hacer referencia directa a las variables `x` y `y`, porque estos son miembros protegidos de la clase base `Point`. Note también que es necesario hacer referencia a `x` y a `y` mediante objetos como en `c.x` y en `c.y`. Esto es debido a que la función del operador de inserción de flujo homónima no es un miembro de la clase `Circle`, sino un amigo de la clase.

El programa manejador (figura 19.4, parte 5) crea `pointPtr` como un apuntador a un objeto `Point` y produce el objeto `p` de la clase `Point`, y a continuación crea `circlePtr`, como un apuntador a un objeto `Circle` y produce el objeto `c` de `Circle`. Los objetos `Point` y `Circle` son extraídos utilizando sus operadores de inserción de flujo homónimos, para demostrar que fueron inicializados en forma correcta. A continuación, el manejador demuestra la asignación de un apuntador de clase derivada (la dirección del objeto `c`) a un apuntador de clase base de nombre `pointPtr` y la conversión explícita (cast) de `pointPtr` de regreso a `Circle` \*. El resultado de la operación de conversión explícita (cast) se asigna a `circlePtr`. El área del objeto `Circle` de nombre `c` es extraído vía `circlePtr`. Esto da como resultado un valor válido de superficie, porque los apuntadores están siempre señalando a un objeto `Circle`. Siempre resulta válido asignar un apuntador de clase derivada a un apuntador de clase base, porque un objeto de clase derivada *es* un objeto de clase base. El apuntador de clase base sólo “ve” la parte correspondiente de clase base del objeto de clase derivada. El compilador lleva a cabo una conversión implícita del apuntador de clase derivada a un apuntador de clase base. Un apuntador de clase base no puede ser asignado en forma directa a un apuntador de clase derivada, porque sería una asignación inherente peligrosa los apuntadores de clase derivada se espera estén apuntados a objetos de clase derivada. En este caso el compilador no llevará a cabo una conversión implícita. El uso de una conversión explícita (cast) informará al compilador que el programador sabe que este tipo de conversión de apuntador es peligrosa y que el programador asume la responsabilidad de utilizar adecuadamente dicho apuntador.

A continuación, el manejador demuestra la asignación del apuntador de clase base (la dirección del operador `p`) al apuntador de clase base de nombre `pointPtr` y la conversión explícita (cast) de `pointPtr`, de regreso a `Circle` \*. El resultado de la operación de conversión explícita (cast) es asignado a `circlePtr`. El radio del objeto al cual `circlePtr` señala (objeto `p` de `Point`) es extraído vía `circlePtr`. Esto da como resultado un valor inválido, porque los apuntadores están siempre señalando a un objeto `Point`. Y un objeto `Point` no tiene un miembro `radius`. Por lo tanto, el programa extraerá cualquier valor que en ese momento exista en memoria en la posición que `circlePtr` espera que esté el miembro de datos `radius`.

19.5 Cómo utilizar funciones miembro

Cuando se forma una clase derivada a partir de una clase base, las funciones miembro de la clase derivada pudiera ser necesario que contengan ciertos miembros de clase base.

Observación de ingeniería de software 19.1

Una clase derivada no puede tener acceso directo a miembros privados de su clase base.

Este es un aspecto crucial de ingeniería de software en C++. Si una clase derivada podría tener acceso a los miembros privados de la clase base, esto violaría el encapsulado de la clase base. El ocultamiento de los miembros privados es gran ayuda en la prueba, depuración y modificación correcta de los sistemas. Si una clase derivada tuviera acceso a los miembros privados de su clase base, sería entonces posible para las clases derivadas de dicha clase derivada también tener acceso a dichos datos, y así en lo sucesivo. Esto propagaría el acceso de lo que se supone deben ser datos privados y a todo lo largo de la jerarquía de la clase se perderían los beneficios del encapsulado.

19.6 Cómo redefinir los miembros de clase base en una clase derivada

Una clase derivada puede redefinir una función miembro de clase base. Cuando en la clase derivada dicha función es mencionada por su nombre, de forma automática se selecciona la versión de la clase derivada. Puede utilizarse el operador de resolución de alcance, para tener acceso a la versión de clase base partiendo de la clase derivada.

Error común de programación 19.3

Cuando se redefine una función miembro de clase base en una clase derivada, es común hacer que la versión de clase derivada llame a la versión de clase base y haga algún trabajo adicional. Causa recursión infinita no utilizar el operador de resolución de alcance para hacer referencia a la función miembro de la clase base, porque la función miembro de la clase derivada, de hecho, está llamando a sí misma.

Práctica sana de programación 19.1

Al heredar capacidades que no son necesarias en la clase derivada, dichas capacidades se deben enmascarar mediante redefinición de las funciones.

Observación de ingeniería de software 19.2

Una redefinición de una función miembro de clase base en una clase derivada no necesariamente debe tener la misma signatura que la función miembro de clase base.

Veamos por ejemplo una clase `Employee` simplificada. Almacena el `firstName` y el `lastName` del empleado. Esta información es común a todos los empleados, incluyendo a aquellos de clases derivadas de la clase `Employee`. De la clase `Employee` derivan ahora las clases `Hourlyworker`, `PieceWorker`, `Boss`, y `CommissionWorker`. Al `Hourlyworker` se le paga por hora, “una vez y media” por las horas extra en exceso de 40 horas por semana. Al `PieceWorker` se le paga una tasa fija por cada elemento producido y para simplificar, suponga que esta persona fabrica un tipo de elemento, por lo que los miembros de datos privados son el número de elementos producidos y la tasa correspondiente por cada elemento. El `Boss` recibe un salario fijo por semana. El `CommissionWorker` recibe un reducido salario base semanal fijo, más un porcentaje fijo de las ventas brutas efectuadas por dicha persona durante la semana. A fin de simplificar, ahora estudiaremos sólo la clase `Employee` y la clase derivada `Hourlyworker`.

Nuestro siguiente ejemplo aparece en la figura 19.5, partes 1 hasta la 5. Las parte 1 y 2 muestran la definición de clase `Employee` y las definiciones de función miembro `Employee`. Las partes 3 y 4 muestran la definición de clase `Hourlyworker` y la definición de función miembro `Hourlyworker`. La parte 5 muestra un programa manejador para la jerarquía de herencia `Employee/Hourlyworker`, que sólo produce un objeto `Hourlyworker`, lo inicializa y llama a la función miembro `print` de `Hourlyworker`, con el fin de extraer los datos del objeto.

```
// EMPLOY.H
// Definition of class Employee
#ifndef EMPLOY_H
#define EMPLOY_H

class Employee {
public:
    Employee(const char*, const char*); // constructor
    void print() const; // output first and last name
    ~Employee(); // destructor
private:
    char *firstName; // dynamically allocated string
    char *lastName; // dynamically allocated string
};

#endif
```

Fig. 19.5 Definición de la clase Employee (parte 1 de 5).

```
// EMPLOY.CPP
// Member function definitions for class Employee
#include <string.h>
#include <iostream.h>
#include <assert.h>
#include "employ.h"

// Constructor dynamically allocates space for the
// first and last name and uses strcpy to copy
// the first and last names into the object.
Employee::Employee(const char *first, const char *last)
{
    firstName = new char[ strlen(first) + 1 ];
    assert(firstName != 0); // terminate if memory not allocated
    strcpy(firstName, first);

    lastName = new char[ strlen(last) + 1 ];
    assert(lastName != 0); // terminate if memory not allocated
    strcpy(lastName, last);
}

// Output employee name
void Employee::print() const
{ cout << firstName << ' ' << lastName; }

// Destructor deallocates dynamically allocated memory
Employee::~~Employee()
{
    delete [] firstName; // reclaim dynamic memory
    delete [] lastName; // reclaim dynamic memory
}
```

Fig. 19.5 Definiciones de funciones miembro para la clase Employee (parte 2 de 5).

```
// HOURLY.H
// Definition of class HourlyWorker
#ifndef HOURLY_H
#define HOURLY_H

#include "employ.h"

class HourlyWorker : public Employee {
public:
    HourlyWorker(const char*, const char*, float, float);
    float getPay() const; // calculate and return salary
    void print() const; // redefined base-class print
private:
    float wage; // wage per hour
    float hours; // hours worked for week
};

#endif
```

Fig. 19.5 Definición de la clase HourlyWorker (parte 3 de 5).

```
// HOURLY_B.CPP
// Member function definitions for class HourlyWorker
#include <iostream.h>
#include <iomanip.h>
#include "hourly.h"

// Constructor for class HourlyWorker
HourlyWorker::HourlyWorker(const char *first, const char *last,
                           float initHours, float initWage)
    : Employee(first, last) // call base-class constructor
{
    hours = initHours;
    wage = initWage;
}

// Get the HourlyWorker's pay
float HourlyWorker::getPay() const { return wage * hours; }

// Print the HourlyWorker's name and pay
void HourlyWorker::print() const
{
    cout << "HourlyWorker::print()\n\n";

    Employee::print(); // call base-class print function

    cout << " is an hourly worker with pay of"
         << " $" << setiosflags(ios::showpoint)
         << setprecision(2) << getPay() << endl;
}
```

Fig. 19.5 Definiciones de funciones miembro para la clase HourlyWorker (parte 4 de 5).

```
// FIG19_5.CPP
// Redefining a base-class member function in a
// derived class.
#include <iostream.h>
#include "hourly.h"

main()
{
    HourlyWorker h("Bob", "Smith", 40.0, 7.50);
    h.print();
    return 0;
}
```

```
HourlyWorker::print()
```

```
Bob Smith is an hourly worker with pay of $300.00
```

Fig. 19.5 Cómo redefinir una función miembro de clase base en una clase derivada (parte 5 de 5).

La definición de clase **Employee** (figura 19.5 parte 1) está formada de dos miembros de datos privados **char \*** que son **firstName** y **lastName** y de tres funciones miembro un constructor, un destructor y **print**. La función constructor (figura 19.5 parte 2) recibe dos cadenas y asigna dinámicamente arreglos de caracteres para almacenar las cadenas. Note que es utilizado el macro **assert** (analizado en el capítulo 14, "Temas avanzados"), para determinar si se asignó memoria a **firstName** y **lastName**. Si no es así, el programa termina con mensaje de error indicando la condición probada, el número de línea en el cual aparece dicha condición, y el archivo en el cual la condición está almacenada. Dado que los datos de **Employee** son **private**, el único acceso a los datos es mediante la función miembro **print**, que sólo extrae el nombre y el apellido del empleado. La función destructor devuelve al sistema la memoria dinámicamente asignada.

La clase **HourlyWorker** (figura 19.5 parte 3) hereda de la clase **Employee** mediante herencia pública. Otra vez, esto queda especificado en la primera línea de la definición de clase, como sigue:

```
class HourlyWorker : public Employee
```

La interfaz pública de **HourlyWorker** incluye la función **Employee print** y las funciones miembro de **HourlyWorker**, de nombre **getPay** y **print**. Note que la clase **HourlyWorker** define su propia función **print**. Por lo tanto, la clase **HourlyWorker** tiene acceso a dos funciones **print**. La clase **HourlyWorker** también contiene los miembros de datos privados **wage** y **hours**, para el cálculo del salario semanal del empleado.

El constructor **HourlyWorker** (figura 19.5 parte 4) utiliza sintaxis de inicializador de miembro para pasar las cadenas **first** y **last** al constructor **Employee**, de forma tal, que puedan ser inicializados los miembros de la clase base, y a continuación inicializa a los miembros **wage** y **hours**. La función miembro **getPay** calcula el salario de **HourlyWorker**.

La función miembro de **HourlyWorker**, de nombre **print**, es un ejemplo de la función miembro de clase base, redefinida en una clase derivada. A menudo se redefinen las funciones miembro de clase base en una clase derivada, a fin de dar más funcionalidad. Las funciones redefinidas a veces llaman a la versión de clase base de la función para llevar a cabo parte de la

nueva tarea. En este ejemplo, la función de clase derivada **print** llama la función de clase base **print**, para extraer el nombre del empleado (la clase base **print** es la única función con acceso a los datos privados de la clase base). La función de clase derivada **print** también extrae la paga del empleado. Note cómo es llamada la versión de clase base de la función **print**

```
Employee::print();
```

Dado que tanto la función de clase base como la función de clase derivada tienen el mismo nombre, la función de clase base debe ser antecedita por su nombre de clase y por el operador de resolución de alcance. De lo contrario, la función correspondiente a la versión de la clase derivada será llamada, lo que resultaría una recursión infinita (la función **HourlyWorker print** se llamaría a sí misma).

## 19.7 Clases base públicas, protegidas y privadas

Al derivar una clase a partir de una clase base, la clase base puede ser heredada como **public**, **protected**, o **private**. La herencia protegida y la herencia privada son raras y deberían ser utilizadas sólo con extremo cuidado; en los ejemplos de este libro solo utilizamos herencia pública.

Al derivar una clase a partir de una clase base pública, los miembros públicos de la clase base se convierten en miembros públicos de la clase derivada, y los miembros protegidos de la clase base se convierten en miembros protegidos de la clase derivada. Los miembros privados de una clase base nunca son accesibles en forma directa desde una clase derivada.

Al derivar una clase a partir de una clase base protegida, los miembros públicos y protegidos de la clase base se convierten en miembros protegidos de la clase derivada.

Cuando se deriva una clase a partir de una clase base privada, los miembros públicos y protegidos de la clase base se convierten en miembros privados de la clase derivada.

## 19.8 Clases base directas y clases base indirectas

Una clase base puede ser una *clase base directa* de una clase derivada, o una clase base puede ser una *clase base indirecta* de una clase derivada. Una clase base directa de una clase derivada es listada en forma explícita en el encabezado de dicha clase derivada cuando se declara esta clase derivada. Una clase base indirecta no se lista en forma explícita en el encabezado de la clase derivada; más bien la clase base indirecta se hereda desde varios niveles arriba en la jerarquía de la clase.

## 19.9 Cómo utilizar constructores y destructores en clases derivadas

Dado que una clase derivada hereda los miembros de su clase base, cuando es producido un objeto de una clase derivada, el constructor de la clase base deberá de ser llamado, para inicializar los miembros de la clase base del objeto de la clase derivada. El constructor de la clase derivada puede llamar de forma implícita al constructor de la clase base o en el constructor de la clase derivada un *inicializador de clase base* (mismo que utiliza la sintaxis de inicializador de miembro que ya hemos visto) puede ser proveído, a fin de llamar de manera explícita al constructor de la clase base.

Los constructores de la clase base y los operadores de asignación de la clase base no son heredados por las clases derivadas. Sin embargo, los constructores y los operadores de asignación de las clases derivadas, pueden llamar a los constructores y a los operadores de asignación de la clase base.

Un constructor de clase derivada siempre llamará primero al constructor correspondiente de su clase base para inicializar los miembros de la clase base de la clase derivada. Si el constructor de la clase derivada ha sido omitido, el constructor por omisión de la clase derivada llamará al constructor de la clase base. Los destructores son llamados en orden inverso a las llamadas de constructor, por lo que un destructor de clase derivada será llamado antes de su destructor de clase base.

#### Observación de ingeniería de software 19.3

*Cuando en una clase derivada se cree un objeto, primero se ejecutará el constructor de clase base, después se ejecutarán los constructores de los objetos miembro correspondientes a la clase derivada, y después se ejecutará el constructor de la clase derivada. Los destructores serán llamados en orden inverso en el cual fueron llamados sus correspondientes constructores.*

El estándar de C++ especifica que el orden en que son construidos los objetos miembros es el orden en el cual han sido declarados dichos objetos dentro de la definición de clase. No afecta a la construcción el orden en el cual los inicializadores de miembro están listados. En la herencia, los constructores de clase base son llamados en el orden en el cual se haya especificado la herencia en la definición de clase derivada. El orden en el cual se han especificado los constructores de clase base en el constructor de clase derivada, no afecta a la construcción.

#### Error común de programación 19.4

*Escribir un programa que dependa de que los objetos miembros sean construidos en un orden particular, puede producir errores sutiles.*

El programa de la figura 19.6 demuestra el orden en que los constructores y destructores de clase base y de clase derivada son llamados. El programa está formado de cinco partes. Las partes 1 y 2 muestran una clase `Point` simple que contiene un constructor, un destructor y miembros de datos protegidos `x` y `y`. Tanto el constructor como el destructor ambos imprimen el objeto `Point` para el cual son invocados. Las partes 3 y 4 muestran una clase simple `Circle` derivada de `Point` mediante herencia pública y que contiene un constructor, un destructor y el miembro de datos privado `radius`. Tanto el constructor como el destructor imprimen el objeto `Circle` para el cual fueron invocados. El constructor `Circle` también invoca al constructor `Point` utilizando sintaxis de inicializador de miembro y pasa los valores `a` y `b`, de tal forma que los miembros de datos de clase base puedan ser inicializados.

En la parte 5 se muestra un programa manejador para esta jerarquía `Point/Circle`. El programa empieza produciendo un objeto `Point` en su propio alcance dentro de `main`. El objeto entra y sale de alcance de inmediato, por lo que tanto el constructor como el destructor `Point` son llamados. El programa a continuación produce el objeto `Circle` de nombre `circle1`. Esto invoca al constructor `Point` para llevar a cabo una salida con los valores pasados a partir del constructor `Circle`, y a continuación ejecuta la salida especificada en el constructor `Circle`. A continuación se produce el objeto `Circle` de nombre `circle2`. Otra vez, tanto el constructor `Point` como el `Circle` son llamados. Note que se ejecuta el cuerpo del constructor `Point` antes del cuerpo del constructor `Circle`. Se alcanza el fin de `main`, por lo que deberán ser llamados los destructores tanto para `circle1` como para `circle2`. Los destructores son llamados en orden inverso a sus constructores correspondientes. Por lo tanto el destructor `Circle` y el destructor `Point` son llamados en este orden para el objeto `circle2`, y a continuación el destructor `Circle` y el destructor `Point` son llamados en ese orden para el objeto `circle1`.

```
// POINT2.H
// Definition of class Point
#ifndef POINT2_H
#define POINT2_H

class Point {
public:
    Point(float = 0.0, float = 0.0); // default constructor
    ~Point(); // destructor
protected: // accessible by derived classes
    float x, y; // x and y coordinates of Point
};

#endif
```

Fig. 19.6 Definición de clase `Point` (parte 1 de 5).

```
// POINT2.CPP
// Member function definitions for class Point
#include <iostream.h>
#include "point2.h"

// Constructor for class Point
Point::Point(float a, float b)
{
    x = a;
    y = b;

    cout << "Point constructor: "
         << '[' << x << ", " << y << ']' << endl;
}

// Destructor for class Point
Point::~~Point()
{
    cout << "Point destructor: "
         << '[' << x << ", " << y << ']' << endl;
}
```

Fig. 19.6 Definiciones de funciones miembro para la clase `Point` (parte 2 de 5).

### 19.10 Conversión implícita de objeto de clase derivada a objeto de clase base

A pesar del hecho que un objeto de clase derivada también "es un" objeto de clase base, el tipo de la clase derivada y el tipo de la clase base son distintos. Los objetos de clase derivada pueden ser tratados como objetos de clase base. Tiene sentido este tipo de asignación, porque la clase derivada tiene miembros que corresponden a cada uno de los miembros de la clase base —recuerde que por lo regular la clase derivada tiene más miembros que la clase base. En la otra dirección la asignación no es permitida, porque asignar un objeto de clase base a un objeto de clase derivada dejaría sin definición a los miembros adicionales de la clase derivada. Aunque tal asignación no

```
// CIRCLE2.H
// Definition of class Circle
#ifndef CIRCLE2_H
#define CIRCLE2_H

#include "point2.h"
#include <iomanip>

class Circle : public Point {
public:
    // default constructor
    Circle(float r = 0.0, float x = 0, float y = 0);

    ~Circle();    // destructor
private:
    float radius;    // radius of Circle
};

#endif
```

Fig. 19.6 Definición de clase Circle (parte 3 de 5).

```
// CIRCLE2.CPP
// Member function definitions for class Circle
#include "circle2.h"

// Constructor for Circle calls constructor for Point
Circle::Circle(float r, float a, float b)
    : Point(a, b)    // call base-class constructor
{
    radius = r;

    cout << "Circle constructor: radius is "
        << radius << " [" << a << ", " << b << "]" << endl;
}

// Destructor for class Circle
Circle::~~Circle()
{
    cout << "Circle destructor: radius is "
        << radius << " [" << x << ", " << y << "]" << endl;
}
```

Fig. 19.6 Definiciones de funciones miembro para la clase Circle (parte 4 de 5).

es permitida en forma "natural", se puede legitimar proporcionando un operador de asignación y/o un constructor de conversión correctamente homónimos.

#### Error común de programación 19.5

Asignar un objeto de clase derivada a un objeto de una clase base correspondiente, y a continuación intentar hacer referencia a miembros de sólo la clase derivada en el objeto nuevo de clase.

```
// FIG19_6.CPP
// Demonstrate when base-class and derived-class
// constructors and destructors are called.
#include <iostream.h>
#include "point2.h"
#include "circle2.h"

main()
{
    // Show constructor and destructor calls for Point
    {
        Point p(1.1, 2.2);
    }

    cout << endl;
    Circle circle1(4.5, 7.2, 2.9);
    cout << endl;
    Circle circle2(10, 5, 5);
    cout << endl;
    return 0;
}
```

```
Point constructor: [1.1, 2.2]
Point destructor: [1.1, 2.2]

Point constructor: [7.2, 2.9]
Circle constructor: radius is 4.5 [7.2, 2.9]

Point constructor: [5, 5]
Circle constructor: radius is 10 [5, 5]

Circle destructor: radius is 10 [5, 5]
Point destructor: [5, 5]
Circle destructor: radius is 4.5 [7.2, 2.9]
Point destructor: [7.2, 2.9]
```

Fig. 19.6 Orden en el cual son llamados los constructores y destructores de clase base y clase derivada (parte 5 de 5).

Un apuntador a un objeto de clase derivada puede ser convertido en forma implícita en un apuntador a un objeto de una clase base, porque un objeto de una clase derivada es un objeto de una clase base.

Existen cuatro formas posibles para mezclar y hacer coincidir apuntadores de clase base y apuntadores de clase derivada con objetos de clase base y objetos de clase derivada:

1. Es simple hacer referencia a un objeto de clase base con un apuntador de clase base.
2. También es simple hacer referencia a un objeto de clase derivada con un apuntador de clase derivada.
3. Es seguro hacer referencia a un objeto de clase derivada con un apuntador de clase base, porque el objeto de clase derivada es también un objeto de su base clase. El programa

sólo puede referirse a miembros de clase base. Si mediante el apuntador de clase base el programa hace referencia a miembros sólo de clase derivada, el compilador informará de un error de sintaxis.

4. Es un error de sintaxis hacer referencia a un objeto de clase base mediante un apuntador de clase derivada. El apuntador de clase derivada deberá ser convertido primero explícitamente en un apuntador de clase base.

#### **Error común de programación 19.6**

*Puede causar error convertir explícitamente (cast) un apuntador de clase base a un apuntador de clase derivada, si a continuación dicho apuntador es utilizado para hacer referencia a un objeto de clase base que no tenga los miembros deseados de la clase derivada.*

Independiente de lo conveniente que resulte tratar los objetos de la clase derivada como si fueran objetos de clase base, y hacerlo manipulando todos estos objetos mediante apuntadores de clase base, existe un problema. En un sistema de nóminas, por ejemplo, nos gustaría ser capaces de recorrer una lista enlazada de empleados y calcular la paga semanal para cada persona. Pero utilizar apuntadores de clase base sólo le permite al programa llamar la rutina de cálculo de nóminas de clase base (si en verdad tal rutina existe en la clase base). Necesitamos una forma para invocar para cada objeto la rutina de cálculo de nóminas adecuada, sea éste un objeto de clase base o un objeto de clase derivada, y hacerlo con facilidad utilizando el apuntador de clase base. La solución es utilizar funciones virtuales y polimorfismo, como se analiza en el capítulo 20.

### **19.11 Ingeniería de software con herencia**

Podemos utilizar la herencia para *personalizar* el software existente. Heredamos los atributos y comportamientos de una clase existente, y a continuación añadimos o retiramos atributos y comportamientos para personalizar la clase a fin de que cumpla con nuestras necesidades. Esto en C++ se efectúa sin que la clase derivada tenga acceso al código fuente de la clase base, pero la clase derivada sí debe tener la capacidad de enlazarse con el código objeto de la clase base. Esta poderosa capacidad resulta atractiva para los fabricantes independientes de software (ISV). Los ISV pueden desarrollar clases propietarias para venta o para licencia, y poner estas clases a disposición de los usuarios en formato de código objeto. Los usuarios pueden entonces con rapidez derivar de estas clases de biblioteca clases nuevas, sin necesidad de tener acceso al código fuente propiedad de los ISV. Todo lo que los ISV tienen que suministrar junto con el código objeto son los archivos de cabecera.

Puede ser difícil para los estudiantes darse cuenta de los problemas a los que los diseñadores y los instaladores se enfrentan tratándose de proyectos de software a gran escala. Las personas con experiencia en estos proyectos invariablemente declararán que, para mejorar el proceso de desarrollo de software, resulta clave fomentar la reutilización del software. La programación orientada a objetos en general, y C++ en particular, alienta y fomenta la reutilización del software.

Es la disponibilidad de bibliotecas de clases sustanciales y útiles la que proporciona el máximo de beneficios en la reutilización del software mediante la herencia. Conforme crezca el interés en C++, aumentará el interés en las bibliotecas de clases. Igual que con la llegada de la computadora personal, el software listo para usarse, producido por fabricantes independientes de software, se convirtió en una industria con un crecimiento explosivo, también, por lo tanto, será la creación y la venta de bibliotecas de clases. Partiendo de estas bibliotecas los diseñadores de aplicaciones construirán sus aplicaciones, y los diseñadores de bibliotecas se verán premiados al empacarse dichas bibliotecas junto con las aplicaciones. Las bibliotecas actualmente incluidas con los

compiladores de C++ tienden a ser más bien de uso general y de alcance limitado. Lo que estamos viendo venir es un compromiso masivo a nivel mundial para el desarrollo de bibliotecas de clases y para una gran variedad de campos de aplicaciones.

#### **Observación de ingeniería de software 19.4**

*Crear una clase derivada no afecta el código fuente o el código objeto de su clase base; mediante la herencia se conserva la integridad de la clase base.*

Una clase base define un estado común. Todas las clases derivadas de una clase base heredan las capacidades de dicha clase base. En el proceso de diseño orientado a objetos, el diseñador busca el estado común y lo extrae para formar clases base deseables. Las clases derivadas después son personalizadas, más allá de las capacidades heredadas de las clases base.

De igual forma que el diseñador de sistemas no orientados a objetos busca evitar una proliferación innecesaria de funciones, el diseñador de sistemas orientados a objetos debería evitar una proliferación innecesaria de clases. Dicha proliferación de clases crea problemas de administración y puede dificultar la reutilización del software, debido a que para el reutilizador potencial de una clase resulta más difícil localizar dicha clase en una gran colección. La contrapartida es crear menos clases, capaz cada una de ellas de proporcionar funcionalidad adicional. Para ciertos reutilizadores dichas clases podrían resultar demasiado ricas; esta funcionalidad excesiva pudiera ser enmascarada, "aminorando" entonces dichas clases a fin de que cumplan con sus necesidades.

#### **Sugerencia de rendimiento 19.1**

*Si las clases producidas mediante la herencia son más grandes de lo requerido, pudiera dar como resultado un desperdicio de memoria y de recursos de proceso.*

Note que puede resultar confuso leer un conjunto de declaraciones de clase derivada, porque no se muestran los miembros heredados. Pero aún así en las clases derivadas los miembros heredados están presentes.

#### **Observación de ingeniería de software 19.5**

*En un sistema orientado a objetos, las clases a menudo están muy relacionadas. "Disgregue y elimine" atributos y comportamientos comunes y colóquelos en una clase base. A continuación utilice la herencia para formar clases derivadas.*

#### **Observación de ingeniería de software 19.6**

*Una clase derivada contiene los atributos y comportamientos de su clase base. Una clase derivada también puede contener atributos y comportamientos adicionales. Con la herencia, la clase base puede ser compilada independientemente de la clase derivada. Para tener la capacidad de combinar estos atributos y comportamientos adicionales con la clase base para formar la clase derivada basta compilar los atributos y comportamientos incrementados de la clase derivada.*

#### **Observación de ingeniería de software 19.7**

*Las modificaciones a una clase base no requieren que sea modificada la clase derivada, siempre y cuando la interfaz pública de la clase base se mantenga sin modificación. Pudiera sin embargo ser necesario recompilar las clases derivadas.*

### **19.12 Composición en comparación con herencia**

Hemos discutido relaciones *es un* basadas en la herencia. También hemos analizado relaciones *tiene un* (y hemos visto ejemplos en capítulos anteriores) en las cuales un objeto puede tener otros

objetos —como miembros mediante la *composición* de clases existentes estas relaciones crean nuevas clases. Por ejemplo, dadas las clases **Employee**, **BirthDate**, y **TelephoneNumber**, no es correcto decir que un **Employee** es un **BirthDate** o que un **Employee** es un **TelephoneNumber**. Pero es correcto decir que un **Employee** tiene un **BirthDate** y un **Employee** tiene un **TelephoneNumber**.

Recuerde que el orden en el cual los objetos miembros se construyen se define como el orden en el cual ha sido declarado dicho objeto. Los constructores de clase base serán llamados en el orden en el cual ha sido especificada la herencia en la clase derivada.

#### Observación de ingeniería de software 19.8

Las modificaciones a una clase miembro no requieren que su clase compuesta que la encierra sea modificada, siempre y cuando se mantenga sin modificación la interfaz pública de la clase miembro. Note que pudiera ser necesario recompilar la clase compuesta.

### 19.13 Relaciones “utiliza un” y “conoce un”

La herencia y la composición, al crear nuevas clases que tengan mucho en común con clases existentes, cada una de ellas fomentan la reutilización del software. Existen otras formas de utilizar los servicios de las clases. Aunque un objeto persona no es un automóvil, y un objeto persona no contiene un automóvil, un objeto persona *utiliza un automóvil*. Una función utiliza un objeto, sólo al emitir una llamada de función a una función miembro de dicho objeto.

Un objeto puede estar *consciente de* otro objeto. Las redes de conocimiento a menudo tienen estas relaciones. Para estar consciente de un objeto, otro objeto puede contener un apuntador o una referencia a dicho objeto. En este caso, se dice que un objeto tiene una relación de conciencia con el otro objeto.

### 19.14 Estudio de caso: Point, Circle, Cylinder

Veamos ahora el ejercicio recapitulativo correspondiente a este capítulo. Veremos una jerarquía de punto, círculo y cilindro. Primero desarrollaremos y utilizaremos la clase **Point** (figura 19.7). Después presentaremos un ejemplo en el cual de la clase **Point** derivaremos la clase **Circle** (figura 19.8). Por último, presentaremos un ejemplo en el cual de la clase **Circle** derivaremos la clase **Cylinder** (figura 19.9).

La figura 19.7 muestra la clase **Point**. La Parte 1 de la figura 19.7 muestra la definición de la clase **Point**. Note que los miembros de datos de **Point** están protegidos. Entonces, cuando de la clase **Point** derivemos la clase **Circle**, las funciones miembro de la clase **Circle** podrán hacer referencia directa a las coordenadas **x** y **y**, en vez de tener que utilizar funciones de acceso. Esto pudiera resultar en un mejor rendimiento.

En la figura 19.7 parte 2, se muestran las definiciones de funciones miembro correspondientes a la clase **Point**. En la figura 19.7 parte 3, se muestra el programa manejador para la clase **Point**. Note que **main** deberá utilizar las funciones de acceso **getX** y **getY** para leer los valores de los miembros de dato protegidos **x** y **y**; recuerde que los miembros de dato protegidos son accesibles sólo a miembros y amigos de su clase y a miembros y amigos de sus clases derivadas.

Nuestro siguiente ejemplo se muestra en la figura 19.8 partes 1 a 3. La definición de clase **Point** y las definiciones de función miembro correspondientes de la figura 19.7 son utilizadas aquí otra vez. Las partes 1 a 3 muestran la definición de clase **Circle**, las definiciones de función miembro **Circle**, y el programa manejador, respectivamente. Note que la clase **Circle** hereda de la clase **Point** mediante herencia pública. Esto significa que la interfaz pública de **Circle**

```
// POINT2.H
// Definition of class Point
#ifndef POINT2_H
#define POINT2_H

class Point {
    friend ostream &operator<<(ostream &, const Point &);
public:
    Point(float = 0, float = 0);        // default constructor
    void setPoint(float, float);        // set coordinates
    float getX() const { return x; }    // get x coordinate
    float getY() const { return y; }    // get y coordinate
protected:                            // accessible to derived classes
    float x, y;                        // coordinates of the point
};

#endif
```

Fig. 19.7 Definición de clase **Point** (parte 1 de 3).

```
// POINT2.CPP
// Member functions for class Point
#include <iostream.h>
#include "point2.h"

// Constructor for class Point
Point::Point(float a, float b)
{
    x = a;
    y = b;
}

// Set the x and y coordinates
void Point::setPoint(float a, float b)
{
    x = a;
    y = b;
}

// Output the Point
ostream &operator<<(ostream &output, const Point &p)
{
    output << '[' << p.x << ", " << p.y << ']'<<endl;

    return output;                // enables concatenation
}
```

Fig. 19.7 Funciones miembro para la clase **Point** (parte 2 de 3).

incluye las funciones miembro **Point**, así como las funciones miembro **Circle** de nombre **setRadius**, **getRadius**, y **area**. Note que la función de operador de inserción de flujo homónimo **Circle** puede hacer referencia directa a las variables **x** y **y**, porque éstas son

```
// FIG19_7.CPP
// Driver for class Point
#include <iostream.h>
#include "point2.h"

main()
{
    Point p(7.2, 11.5);    // instantiate Point object p

    // protected data of Point inaccessible to main
    cout << "X coordinate is " << p.getX()
         << "\nY coordinate is " << p.getY();

    p.setPoint(10, 10);
    cout << "\n\nThe new location of p is " << p << endl;

    return 0;
}
```

```
X coordinate is 7.2
Y coordinate is 11.5

The new location of p is {10, 10}
```

Fig. 19.7 Manejador para la clase Point (parte 3 de 3).

miembros protegidos de la clase base **Point**. Note también que es necesario hacer referencia a **x** y a **y** mediante objetos, como en **c.x** y en **c.y**. Esto es debido a que la función de operador de inserción de flujo homónima no es una función miembro de la clase **Circle**, pero un amigo de la clase. El programa manejador produce un objeto de la clase **Circle**, a continuación utiliza funciones *get* para obtener la información sobre el objeto **Circle**. Otra vez, **main** no es una función miembro ni amigo de la clase **Circle**, por lo cual no puede hacer referencia directa a los datos protegidos de la clase **Circle**. El programa manejador utiliza entonces la función *set* de nombre **setRadius** y **setPoint** para redefinir el radio y las coordenadas del centro del círculo. Por último, el manejador hace algo en especial interesante. Inicializa la variable de referencia **pRef** del tipo "referencia a un objeto **Point**" (**Point &**), al objeto **c** de **Circle**. El manejador a continuación imprime **pRef**, el cual, a pesar del hecho que es inicializado con un objeto **Circle**, "piensa" que es un objeto **Point**, por lo que el objeto **Circle** de hecho se imprime como un objeto **Point**.

Nuestro último ejemplo se muestra en la figura 19.9, partes 1 a 3. Las definiciones de la clase **Point** y de la clase **Circle**, y las definiciones de funciones miembro correspondientes a las figuras 19.7 y 19.8 son vueltas a usar aquí. Las partes 1 a 3 muestran la definición de clase **Cylinder**, las definiciones de función miembro **Cylinder**, y un programa manejador, respectivamente. Note que la clase **Cylinder** hereda de la clase **Circle** mediante herencia pública. Esto significa que la interfaz pública a **Cylinder** incluye las funciones miembro **Circle**, así como las funciones miembro **Cylinder**, de nombre **setHeight**, **getHeight**, **area** (redefinidos a partir de **Circle**) y **volume**. Note que la función de operador de inserción de flujo homónima **Cylinder** es capaz de hacer referencia directa a las variables **x**, **y**, y **radius**, porque

```
// CIRCLE2.H
// Definition of class Circle
#ifndef CIRCLE2_H
#define CIRCLE2_H

#include "point2.h"

class Circle : public Point {
    friend ostream &operator<<(ostream &, const Circle &);
public:
    // default constructor
    Circle(float r = 0.0, float x = 0, float y = 0);
    void setRadius(float);    // set radius
    float getRadius() const;  // return radius
    float area() const;      // calculate area
protected:    // accessible to derived classes
    float radius;    // radius of the Circle
};

#endif
```

Fig. 19.8 Definición de la clase Circle (parte 1 de 3).

```
// CIRCLE2.CPP
// Member function definitions for class Circle
#include <iostream.h>
#include <iomanip.h>
#include "circle2.h"

// Constructor for Circle calls constructor for Point
// with a member initializer and initializes radius
Circle::Circle(float r, float a, float b)
    : Point(a, b)    // call base-class constructor
{ radius = r; }

// Set radius
void Circle::setRadius(float r) { radius = r; }

// Get radius
float Circle::getRadius() const { return radius; }

// Calculate area of Circle
float Circle::area() const
{ return 3.14159 * radius * radius; }

// Output a circle in the form:
// Center = [x, y]; Radius = #.##
ostream &operator<<(ostream &output, const Circle &c)
{
    output << "Center = [" << c.x << ", " << c.y
         << "]; Radius = " << setiosflags(ios::showpoint)
         << setprecision(2) << c.radius;

    return output;    // enables concatenated calls
}
```

Fig. 19.8 Definiciones de función miembro para la clase Circle (parte 2 de 3).